EurekaZero

Engineered Intelligence for Algorithm Invention by Self-Play

JopeDime Team

Draft — November 2026

Abstract

[TBD: Supply the problem statement, proposed method, supported task scope, contributions and evaluation summary from Jepson's source.]

[TBD: State the headline first-proposal result with its uncertainty once the independent evaluation is complete.]

1 Can Machines Invent?

We specify how to test whether feedback from execution improves the first algorithm a system proposes for a new problem. AlphaGo Zero supplies a precedent within a fixed game: it went beyond the best human play without seeing a human game, learning from self-play under the rules of Go [101].

The skeptics' challenge. Language models retrieve existing solutions rather than find new ones, Yann LeCun argued in March 2025: "in their purest form, they are incapable of inventing new things" [58]. Thomas Wolf warned that current trends will give us "a country of yes-men on servers" [117].

Dwarkesh Patel had asked Dario Amodei in August 2023 what to make of models with "basically the entire corpus of human knowledge memorized" that had not made "a single new connection that has led to a discovery" [80]. In January 2025, Demis Hassabis posed the hardest version as a benchmark for general intelligence: today's systems "can play a game of Go at a world champion level. But could a system invent Go?" [41]. These challenges concern both proposing solutions and choosing new questions; our test concerns only candidate algorithm proposals under human-defined tasks and evaluation metrics (Section 8).

Predicting historical text does not directly reward improvement over an incumbent. That motivates a different training signal; it does not establish a ceiling on what a pretrained model can discover (Section 2).

The human challenge. Research effort rose while productivity declined in the industries, products and firms examined in the 2020 study by Bloom et al. [8]. Their Moore's law example required more than 18 times as many researchers as in the early 1970s to sustain chip-density growth. Jones [46] found later first inventions and greater specialization in the inventor data he studied. Park et al. [79] reported declining disruption on citation-based measures of papers and patents. These observations motivate our question: can execution feedback help a proposer depart usefully from familiar ideas? Section 2.4 develops the role of the scientific record.

From search to learning. Tests can select a better candidate without teaching the next proposer. Automated research systems already couple proposals to tests [37, 49, 65, 75, 104, 105]. Sections 2.5 and 3 distinguish retained candidate solutions from learned proposer updates.

The Zero move. AlphaGo learned from human expert games and searched heavily at every move [100]. AlphaGo Zero learned only from games against itself, and its network alone, with no search, rated 3,055 Elo. That is close to the 3,144 of AlphaGo Fan, the version that beat the European champion Fan Hui [100, 101].

This motivates an analogous goal for invention: learn from repeated program evaluations during training, then deploy a writer with a single-candidate budget, search $= 1$ (Box 1). The budget is a test condition, not a promise that every first proposal wins. Figure 1 separates the training feedback path from the frozen writer and fitting procedure at deployment.

Engineered intelligence. The reference design assigns each role in Section 2 an inspectable component: a declared task and metric, a grammar-constrained writer, a referee and retained state (Table 1). Program-incumbent promotion and proposed writer admission are distinct rules in that design. We call the design *engineered intelligence*: its aim, proposer, referee and retained state are specified for audit. Pretrained components still supply prior knowledge (Box 1); execution feedback supplies the additional teacher.

Contributions. We formulate algorithm invention as a game and give a tabular-regression reference whose moves are typed programs of learning primitives.

Key idea

Can judgment bought during training improve the first proposal at deployment?

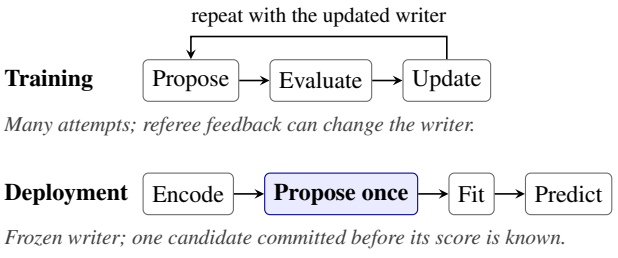


Figure 1: **Training uses feedback; deployment commits one move.** Reference schematic: the proposed procedure updates the writer during training and fits the emitted estimator during deployment. The adapter-only record in Section 6.8 is a separate loop; first-proposal improvement remains the empirical question in Section 7.

Box 1: What "search = 1" means

- **One inference, one candidate.** For each new problem, the program writer emits one complete program ($B = 1$), with no retries, repairs, best-of-N selection, reranking or parameter weight updates.
- **Everything is counted.** This includes reasoning tokens and problem encoding. One inference is one generation, not one forward pass.
- **Declare the fitting procedure.** Fitting the emitted algorithm to training rows is allowed. Any internal model or hyperparameter selection must be disclosed and counted in its compute budget. One outer proposal does not establish the absence of internal search. Training search is priced separately in Section 5.

A single candidate, with prior knowledge: human-designed program primitives and pretrained encoding components supply prior knowledge. The Zero analogy concerns the additional training feedback from the referee.

1. **The anatomy of innovation:** four pillars (aim, variation, judgment and retention), with an idealized relation between proposal quality and the number of attempts (Section 2).
2. **The innovation game:** a board, move, referee and opponent, with a reward ordering and two reference admission rules; each task type requires a qualified board encoding and referee (Section 4).
3. **The economics of algorithmic self-play:** a cost model for self-play training and for the expected cost per verified improvement (Section 5).
4. **EurekaZero:** a reference learner with a frozen problem encoder, a grammar-constrained program writer and a problem–program ranking map, plus an expert-iteration training procedure and audit safeguards against referee reward hacking (Section 6).
5. **An evaluation protocol for the single-proposal claim:** paired comparisons with frozen algorithms whose receipts record the model that designed them, performance curves against agents allowed repeated attempts and a test of the map's ranking against a program-only control (Section 7).

Evidence scope. The adapter records in Section 6.8 establish neither neural writer updates nor a training gain in the separate ranking map. Independent first-proposal comparisons remain pending; Sections 6 and 7 specify the full reference design and its evaluation tests.

Sections 2 and 3 identify the roles and components; Section 4 defines an attempt. Sections 5 and 6 account for its cost and specify how attempts supply reference training targets. The empirical tests in Section 7 then bound the scope and conclusions in Sections 8 and 9.

2 The Anatomy of Innovation

The learning question in Section 1 requires separating four roles: aim, variation, judgment and retention. They distinguish a system's proposals, evaluation and retained experience. We call judgment learned before a try *taste*.

2.1 Novelty is free

Novelty is the cheapest part of invention. Borges imagined a library of every possible book of 410 pages [11]. Every algorithm short enough to fit in a volume is already on its shelves. Nothing in the library tells a reader which volume is true. A random string generator is the same library with a faster librarian: almost every string it emits is new, and almost none is worth running (Figure 2, lower right).

Language-model studies test this distinction. In a blinded study with 104 NLP researchers, ideas from a language model were rated more original than experts' ideas ($p < 0.05$) [98]. Once 43 experts had spent over 100 hours each carrying out ideas from both pools, the generated ideas lost significantly more ground on every metric [99].

Boden had predicted the asymmetry in 1998: machines, she wrote, would find generating new ideas easier than automating their evaluation [10]. The two studies motivate a distinction between the novelty of a proposal and its value after execution; they do not establish that judgment is the sole bottleneck.

2.2 The four pillars

The four pillars provide a common vocabulary for comparing accounts of discovery from biology, psychology, philosophy and engineering (Table 1). This organization is our synthesis; the underlying theories remain distinct.

Campbell argued that every genuine gain in knowledge needs a mechanism for introducing variation, a consistent selection process and a mechanism for preserving and reproducing what is selected [15, p. 381]. The aim is what keeps the selection consistent, which makes it the fourth pillar.

Aim fixes the problem and what counts as better. Hamming asked colleagues, "What are the important problems of your field?" [40]. Pólya's first step is to understand the problem [85], and Popper argued that observation is always selective, so it presupposes a problem [87].

Variation proposes candidates. Peirce called abduction "the only logical operation which introduces any new idea" [81,

Table 1: **Traditions of discovery share four recurring roles.** A dash marks a pillar outside a tradition's scope: Wallas analyzed a single act of thought. For Darwin and Campbell, the aim is a criterion set by the environment. Engineering is the design–build–test–learn cycle [74]; AlphaGo Zero follows Silver et al. [101]; EurekaZero is the reference design (Sections 4 and 6).

Tradition	Aim	Variation	Judgment	Retention
Darwin [25]	Fitness to the environment	Variation	Natural selection	Inheritance
Wallas [115]	Preparation	Incubation, illumination	Verification	—
Campbell [15]	A selection criterion	Blind variation	Selection	Retention
Popper [87]	The problem	Conjecture	Refutation	The surviving theory
The scientific method	Question	Hypothesis	Experiment	Publication
Engineering	Design target	Design and build	Test	Learn
AlphaGo Zero	Win the game	Moves chosen with search	Game outcome	Network weights
EurekaZero	Problem encoding and metric	Program writer	Referee on hidden folds	Incumbent pool; proposed writer updates

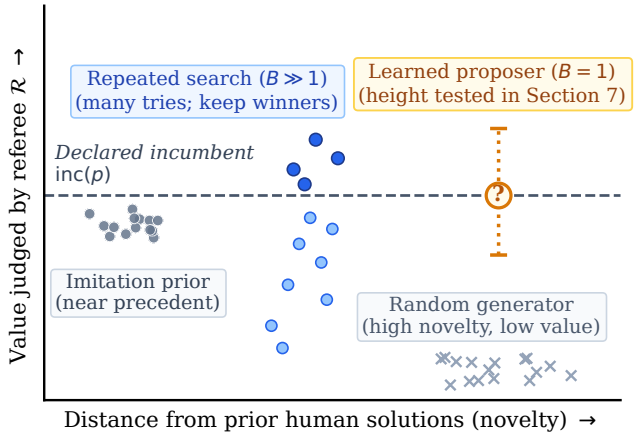


Figure 2: **Novelty is free; judgment is the inventor.** Schematic: horizontal axis shows distance from what people have already done; vertical axis shows value as judged by the referee. A random generator travels far and finds little. Imitation, repeated search and a learned proposer illustrate different ways to select candidates; their positions are conceptual, not measured limits. The value of EurekaZero's single proposal remains an empirical question for Section 7.

CP 5.171]. Campbell modeled creative thought as blind variation [15, 103]. Koestler located creative acts in bisociation, joining two unrelated frames of thought [52]. Boden named three routes to a new idea: combining familiar ideas, exploring a conceptual space and transforming it [10].

Judgment tests a candidate against something outside the mind that proposed it: the attempted refutation [86], the experiment, peer review and the referee of the common task framework, which scores every submitted entry on held-out data that the competitors never see [29].

Retention preserves what survives: inheritance for Darwin [25], selective retention for Campbell [15], the textbook for a science and the incumbent for a game. It can store either an output or a way to propose better outputs. Archiving a winning program does the first; using evaluation feedback to update the proposer attempts the second. This distinction connects the four pillars to the learning question in Section 1; Table 2 compares where human and machine systems retain feedback.

2.3 Taste is judgment moved upstream

Skill in invention is judgment that arrives before the try. For Poincaré, creating meant not making the useless combinations: "Invention is discernment, choice." The sterile combinations, he added, do not even present themselves to the mind of the inventor [83, pp. 324–325].

Chess masters show the same economy: in de Groot's think-aloud studies they considered about as many moves as weaker players but found the right ones [26]. Chase and Simon studied chunks through perception and recall tasks [18]. Much of that knowledge is tacit; experts know more than they can say [84]. AlphaGo's policy network first learned to select candidate moves from human expert games, and then refined that move judgment through self-play [100].

Taste has a price we can state. Suppose each try beats the incumbent with a fixed, nonzero probability q, independently of the others. The number of tries up to and including the first win is then geometric, with mean

$$\mathbb{E}[N_{\text{tries}}] = \frac{1}{q}, \tag{1}$$

where N_{tries} counts all candidate attempts, including incomplete or invalid outputs and failed executions. Under this fixed-probability, independent-try model, better proposal quality shortens the expected search.[1]

The equation relates proposal quality to expected attempts, whether training, structural constraints or revealed outcomes improve that quality. Comparisons of random generators, pretrained models and agent systems therefore need measured win probabilities. In particular, search = 1 fixes the number of candidates a system may submit; it does not require the probability of a win to approach certainty. A single-candidate system can fail, and every failure must remain in its reported

[1] Adaptive search and correlated proposals need not follow Equation (1); independence alone does not say whether the approximation overstates or understates their cost. Empirical attempt curves are required for those systems.

results. Section 5 uses the same idealization to relate evaluation cost to the cost of a verified improvement.

Key idea

Novelty is free. Judgment is the inventor.

2.4 Learning from a referee

People learn much of their taste from the record of their field. Normal science, in Kuhn's account, solves puzzles inside a reigning paradigm [55]. The record also holds people in place: when a star life scientist dies prematurely, articles by outsiders to the star's collaborators flow into the field, draw upon different prior literature and are disproportionately likely to be highly cited [2].

A model trained to predict text receives a signal about what was written, rather than a direct score for improving an algorithm. That does not prevent novel generalization. It motivates supplementing the historical record with feedback from execution, so that a useful departure from historical precedent can be rewarded.

A referee offers such an additional teacher. AlphaGo Zero learned with no human data, from the rules and the outcomes of its own games, and found strategies outside traditional Go knowledge [101]. Its predecessor had already made such a move. As David Silver told Wired in March 2016, AlphaGo put the chance that a human would play its Move 37 against Lee Sedol at one in ten thousand, and it won that game [69].

The knowledge then flowed back to people. In more than 5.8 million moves by professionals from 1950 to 2021, previously unseen moves grew more frequent once superhuman programs arrived, and came with better decisions [97]. These results establish the possibility in Go. Whether an analogous execution-feedback signal improves algorithm invention must be measured directly in the innovation game.

2.5 Where the pillars live

Machines distribute the pillars across different components (Table 2). Swarm designs fit the same anatomy. In an October 2026 essay drawn from his book, Socher rests the Eureka Machine on "a living map of all human knowledge", "a unified model of physical reality", "high-fidelity digital twins" and "fully autonomous physical laboratories" [104, 105]. On top of them, millions of AI agents critique each other's ideas. In our terms, that knowledge archive supplies retention; the model of reality, the twins and the laboratories supply judgment; and the agents supply variation and critique.

Agent search can retain programs, revise its context or update its proposer. The label *swarm* does not specify which it does. The comparison in Table 2 isolates deployment with fixed proposer weights; EurekaZero aims to improve the proposer through training before a new problem arrives. Both need a reliable referee, and training costs belong in any account of deployment savings.

Table 2: **Each approach retains feedback in a different place.** The agent-search column describes fixed-weight deployment loops, not every agent system. The EurekaZero column states the proposed training and deployment protocol. Across every column, human researchers still set the aim.

Aspect	Human science	Agent search	EurekaZero
Aim	The researcher chooses	A person writes the goal	Problem encoding and metric, chosen by people for now
Variation	The scientist's ideas	Agents propose and critique	One candidate per test problem
Judgment	Experiments, peer review	Experiments, simulators, autonomous labs	Referee on hidden folds
Retention	The literature	Logs, archives, a map of knowledge	Incumbent pool; proposed writer updates
Where taste lives	Years of training, in one head	Weights fixed; context can adapt	Writer training (proposed)
Deployment budget	Varies by problem	Multiple adaptive attempts	One candidate; success measured

Where feedback is retained distinguishes these systems. Section 3 traces the AI components that connect proposals to executable tests and possible proposer updates.

3 The Road to Zero: Five Building Blocks

Five lines of AI research connect the four pillars of Section 2: deep learning, program search, reinforcement learning, games and the race for self-improvement. Each contributes a different way to propose, test or retain a candidate. Table 3 compares their outputs, deployment budgets and retained experience.[2]

3.1 Deep learning gave variation a voice

Deep learning built the variation pillar: machines that propose. Models learned representations from data [59]. AlexNet made the case on images [54], the Transformer modeled sequences with attention [112], and empirical scaling laws related language-model loss to model size, dataset size and training compute [42, 48]. Sutton [108] drew the lesson: general methods that use computation win.

Generative models turned representations into proposals in text, video [14] and code. Codex solved 28.8% of the HumanEval programming problems with one sample, and 70.2% with 100 samples, counting any that passed the unit tests [19]. That gap illustrates the value of a larger sampling budget. It is not a direct estimate of the geometric model in Equation (1): pass rates and the probability of beating an incumbent measure different outcomes.

Learned prediction and selection also support scientific discovery: protein structures that rival experiment in most CASP14 cases [47], 2.2 million crystal structures predicted

[2]Each history keeps only the milestones the argument needs; Appendix A collects the referees those milestones rely on.

stable relative to the Materials Project [68], and halicin, an antibiotic that then treated infections in mice [106].

CLIP placed images and captions in a single shared space [88], an analogy for our problem–program ranking map (Section 6.4). The reference objective supervises the map with programs scored within each problem. During training search, its ranks can guide parent selection, yet they neither condition nor screen the writer; consequently, map-ranking accuracy and first-proposal quality require separate tests.

Executable tests connect generated programs to feedback. EurekaZero proposes training on that feedback, then measuring first-proposal quality beyond its training tasks.

3.2 Program search built the variation–judgment loop

Program evolution joined variation to judgment: propose a program, score it, keep the better ones. Friedberg [35] let a machine vary its own instructions, Holland [43] made variation and selection a general method of adaptation, and Lenat's AM and EURISKO searched concepts and heuristics by measures of interest [63, 64]. EURISKO takes its name from the Greek verb behind *eureka*, as ours does. Koza [53] closed the loop with genetic programming (GP): a population of programs, a fitness function as referee and selection as coach. Populations retain individual programs within a run, whereas learned proposers can also retain reusable patterns for later tasks.

Subsequent work sharpened both variation and judgment: superoptimization [67], example-driven program synthesis [38] guided by neural networks [5], neural architecture search [124], and AutoML-Zero, which evolved complete learning algorithms from basic mathematical operations [89].

Language models then became the variation operator. FunSearch found cap-set constructions beyond the best known [91]: verified new solutions within a fixed task, bought with search. Lion, an optimizer discovered by program search, entered production use at Google [20]. AlphaEvolve multiplied two 4×4 complex matrices with 48 scalar multiplications, improving on Strassen's algorithm in that setting after 56 years [75], and MLEvolve extended the same search loop to machine-learning engineering tasks [30].

EurekaZero adopts executable programs as moves inside this generate–judge loop, and asks how much of a verified search can transfer into the proposer itself. DreamCoder already learns reusable libraries and a search-guiding model across tasks [31]; a single-candidate evaluation tests how useful a learned proposal is before additional task-specific search.

3.3 Reinforcement learning turned retention into taste

Program search retains solutions; policy updates change the proposer. Reinforcement learning uses the referee's verdicts for those updates. Bellman [6] gave it an equation of value. Samuel's checkers program improved by playing a copy of itself [93], TD-Gammon reached near the best human backgammon players by self-play [109], and DQN learned Atari games from pixels [70]. AlphaGo learned from human games and searched at every move [100]; AlphaGo Zero learned by self-play alone [101]. AlphaZero mastered three games with one algorithm [102], MuZero learned a model for planning [95], and AlphaStar's league of mutually exploiting agents reached Grandmaster level in StarCraft II [113].

In expert iteration [1], search is the teacher and the network is the student (Box 2). Language models also learn from feedback: human preferences [22, 78] and rule-based checks of answers, as in DeepSeek-R1 [39].

This recipe needs a game whose referee compares algorithm performance. The next line of work shows how tasks acquire rules and executable referees.

3.4 Gamification made judgment cheap

Games built the judgment pillar at scale. Shannon [96] chose chess because its moves and its goal are sharply defined. The common task framework made machine-learning research a game: shared data, enrolled competitors and a referee who scores every entry on a sealed test set [29], as in the Netflix Prize [7] and ImageNet [92]. People played science too. Foldit players beat automated methods on hard protein puzzles [23] and solved a protease structure that earlier attempts had not [50]; EteRNA players, whose designs were tested in a laboratory, found RNA design rules [60].

Then machines played problems. AlphaTensor's TensorGame beat the best-known matrix-multiplication algorithms for several sizes [33], and AlphaDev's AssemblyGame found sorting routines now in the LLVM C++ library [66]. AlphaGeometry learned olympiad geometry without human demonstrations [110]. AlphaProof, using the Lean proof checker, and AlphaGeometry 2 jointly reached silver-medal standard on the 2024 International Mathematical Olympiad problems, using computation over multiple days [45].

These benchmarks supply the recipe for a game (clear rules, a referee, a score and a sealed test) and raise the question of which parts of a task can vary under a shared representation and referee. EurekaZero's reference design makes the problem an input to the proposer (Section 4); whether that supports transfer across families remains a result to establish.

3.5 The race for self-improvement scaled search

The race for self-improvement spans program search, agent scaffolds and learned policies. Good [36] foresaw an "intelligence explosion" from recursive design, and Schmidhuber [94] formalized a Gödel machine that rewrites its own code once it proves the rewrite useful.

Box 2: Self-play in 90 seconds

In AlphaGo Zero, a player starts with the rules and plays itself. A short search before each move finds better moves than instinct would, and the referee scores each finished game. The network learns to propose the moves the search chose (its policy) and to predict the winner (its value), so the next search is stronger. Over millions of games, search is distilled into intuition [1, 101]. *What if the board were a problem, and the move an invention?*

Table 3: **Related approaches retain experience in different forms.** This qualitative comparison separates output, deployment budget and retained state. Exact budgets depend on each study's protocol. The last row states EurekaZero's design; its proposal quality is still to be tested.

System	Output	Deployment budget	Retained experience	Feedback source
Genetic programming; AutoML-Zero	An algorithm	Study-specific search	Selected programs	Executable objective
AlphaTensor; AlphaDev	An algorithm for one family	Task-specific search	A policy for that game	Game reward
FunSearch; AlphaEvolve	An algorithm	Iterative search	Program archive	Automated evaluation
Research swarms (AI Scientist, autoresearch)	Experiments and code	Adaptive attempts	Context, logs and archives	Experimental outcomes
EvoTune; TTT-Discover; Evolution Fine-Tuning	An algorithm or construction	Protocol-specific search	Fine-tuned policy or search state	Search trajectories
TabPFN; MotherNet; KumoRFM	Predictions or network weights	Task-specific inference	Pretrained parameters	Training across tasks
Absolute Zero; STP	Answers and proofs	Decoding budget varies	Policy parameters	Correctness feedback
EurekaZero	An algorithm	One candidate	Proposed writer updates	Referee scores ranked within each problem

In practice, STOP improves the scaffold that calls a language model, including itself, though its authors note: "Since the language models themselves are not altered, this is not full recursive self-improvement" [120]. Similarly, the Darwin Gödel Machine's self-editing agents raised its SWE-bench score from 20.0% to 50.0% [121], and AlphaEvolve cut Gemini's training time by 1% [75].

Language models also play themselves. SPIN plays its previous iteration [21], Absolute Zero proposes and solves coding tasks under a code executor [122], and STP pairs a conjecturer with a Lean prover [28]. Bailey et al. [3] attribute plateaus in LLM self-play to reward hacking by the conjecturer and evaluate self-guidance in formal theorem proving.

Our closest neighbors learn to discover. EvoTune fine-tunes on what its own evolutionary search finds [107], and TTT-Discover trains on the one problem at hand, at a few hundred dollars per problem [119]. Evolution Fine-Tuning learns from 156,000 search trajectories across 371 tasks [61], and DiscoRL meta-learns a reinforcement-learning rule that beat human-designed ones [77]. These approaches retain improvements in different places: a search trajectory, a task-specific learner or a reusable learning rule. Our comparison asks about the quality and cost of the first complete candidate, rather than treating these forms of learning as absent.

Research itself is becoming a swarm. The AI Scientist automates ideas, experiments and papers [65], while the AI co-scientist ranks agents' hypotheses in an Elo tournament [37]. Karpathy's autoresearch, released in March 2026, runs about 100 experiments overnight and keeps what improves [49], and in May 2026 Recursive raised $650 million to build AI that improves itself [13]. Under the innovation game's formulation, an encoded research task serves as the board and its evaluator serves as the impartial referee (Section 4).

Self-improvement comes in three strengths: a system can find better external algorithms, build better components of itself or improve its own ability to improve, raising verified discoveries per unit of compute across generations. Each requires separate evidence (Section 8); a recorded training trajectory alone establishes none of them on an unseen task.

Together, these systems motivate two reference requirements: verify an improvement before retaining it, and sandbox candidate execution (Section 6.6). The proposed learning test compares a trained writer against its frozen reference under the same deployment budget, separately from the adapter operating record in Section 6.8.

3.6 Where the roads meet

The comparison in Table 3 requires matching what a system produces and what it spends. Cross-task predictors and network generators [34, 44, 73] differ from readable-program output; one candidate does not equalize compute or inputs. Absolute Zero and STP use correctness feedback, whereas a declared incumbent sets our comparison standard. Section 4 defines the resulting game; Figure 3 separates reference training from game-day evaluation.

Key idea

The test is whether experience on earlier discovery tasks improves the first complete proposal on a new one.

4 Innovation Is a Game

An invention attempt is the reference game's unit of play (Section 3): the encoded problem is the board, a program is the move, the referee checks and scores it, and a declared incumbent is the opponent. The encoding supplies a shared interface; each task type still needs its own qualified encoding, language and evaluation protocol (Section 4.5). Table 4 makes the comparison with Go explicit.

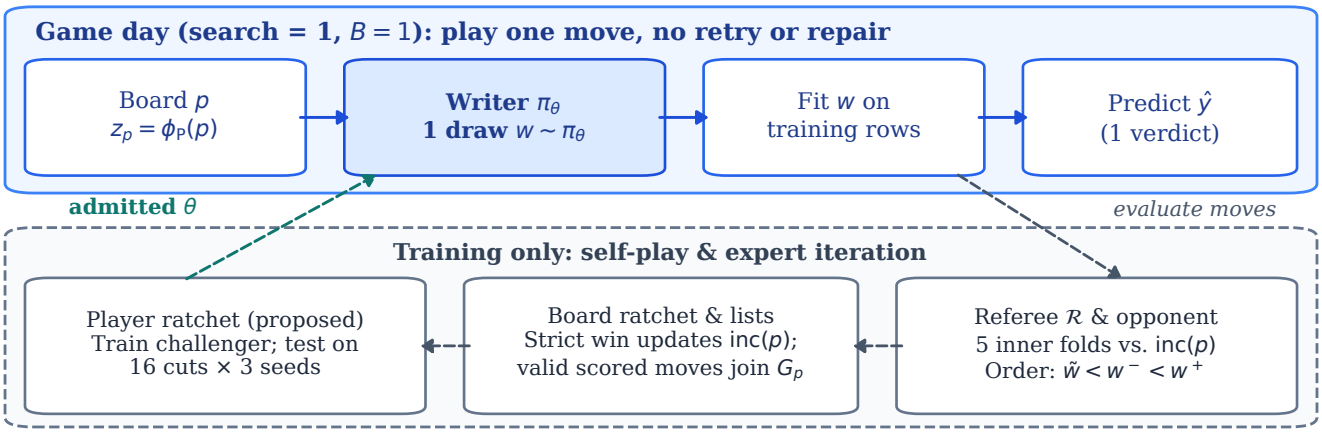


Figure 3: **Reference training searches many moves; game-day evaluation commits one.** In the top band (blue, search = 1), the frozen encoder reads board p and the writer π_θ commits one move w with no retry or repair. In the lower dashed box (proposed training, Section 6.5; separate from the adapter record in Section 6.8), valid scored moves join G_p while failed attempts consume budget.

4.1 Why games came first

Games became AI's laboratory because they supply four things at once: fixed rules, an impartial referee, a clear winner and cheap replay. Chess was chosen because it is "sharply defined both in allowed operations (the moves) and in the ultimate goal (checkmate)" [96], and checkers showed that a program could improve by playing itself [93].

Together, these four properties support repeated, reproducible evaluation. People scale judgment the same way, through prizes, leaderboards and the common task framework's "scoring referee" [29] (Section 3). Invention shares the first three properties once the problem and solution are specified, within the limits of Section 4.6; whether replay becomes cheap is a question of evaluation cost (Section 5).

4.2 The rules

Every submitted program counts as an attempt, including an illegal move. The reference writer π_θ reads the encoded problem, while the referee keeps the incumbent as a separate comparison standard (Section 6.2); its code and score must stay outside the writer's input.

Figure 3 separates training from game day. Its top row follows the single-candidate contract in Box 1.

Formally, the game is a tuple of seven parts,

$$\mathcal{G} = (\mathcal{P}, \phi_P, \mathcal{S}, \phi_S, \mathcal{R}, r, \mathcal{O}).$$

In the reference learner, ϕ_P represents a problem $p \in \mathcal{P}$ as z_p, while ϕ_S embeds a program $w \in \mathcal{S}$ as z_w for the separate ranking map (Section 6.4). Game-day generation and the referee do not use ϕ_S. The referee $\mathcal{R}$ reports validity and score, the reward r orders moves, and the opponent $\mathcal{O}$ supplies the incumbent inc(p). The tuple extends Rice's model of algorithm selection [90]: the player writes the algorithm instead of picking it, and a declared incumbent sets the comparison standard.

With one move, the game is a contextual bandit: the board supplies context, the program is the action and the verdict is the reward [57]. The incumbent supplies a changing comparison standard as verified programs displace it. The asymmetry is in the moves: Go offers about 250 legal moves per turn over about 150 turns, each free to check [100], while this game offers one move, costly to judge. Under a single-candidate budget, the chance q of a win matters directly (Equation (1)). Whether training raises that win probability on unseen problem families is the empirical test in Section 7.

Table 4: **Go and the innovation game share structure but differ in move cost.** Go's moves are cheap, while each program is costly to judge (Section 4.6). Go's game length is from Silver et al. [100].

Element	Go	The innovation game
Board	Stones on a 19×19 grid	The encoded problem $z_p = \phi_P(p)$ for $p \in \mathcal{P}$
Move	Place one stone, or pass	Write one complete program $w \in \mathcal{S}$
Legal move	Empty point; no suicide or forbidden repetition	Parses, type-checks, runs within limits
Illegal move	Not allowed	Stillborn ($\tilde{w}$): an automatic loss
Referee	The rules of Go	The referee $\mathcal{R}$, on hidden folds
Win	Higher final score	Valid move w^+ beating inc(p) under r
Opponent	A past self	The incumbent inc(p) from $\mathcal{O}$, often a past self
Rating	Elo	Win rate and Elo across problems
Moves per game	About 150, each cheap	One (B), costly to judge

4.3 The board and the move

The board is the aim pillar of Section 2, made machine-readable: a summary of the training data, the objective and

metric, and the constraints. Landmark scores from cheap learners [82] and learned tabular summaries are possible encodings; Section 6.2 specifies the reference encoder and its cost limit. In the inspected implementation, that encoder consumes prepared numeric training rows and targets; the writer's engine and regression-task tokens are supplied separately. The objective, metric and budget are protocol settings, not additional fields in this encoder call.

A useful board should preserve distinctions relevant to algorithm choice, remain stable under irrelevant reorderings, exclude final evaluation information and have a measured cost. These are requirements to test, not properties guaranteed by naming an encoder. People still choose the problem and define what counts as a useful solution.

A move is one complete program written in a typed language over learning primitives. A task-specific grammar restricts legal moves, and Section 6.3 specifies the reference operator vocabulary and composition limits.

Types remove whole classes of stillborn programs before they run [71]. Canonicalization can merge syntactic variants; it need not identify every functionally equivalent implementation. AutoML-Zero reused scores when hashes of truncated predictions matched after a short training and validation probe [89]. Matching a probe does not establish agreement on other inputs (Section 6.3).

The language bounds what can be expressed; changing its primitives changes that space [9]. Structural constraints can prevent invalid programs; their effect on useful proposal quality must be measured (Section 7).

4.4 The referee and the opponent

The reward is an ordering first and a number second. Validity is a gate: any stillborn program scores below any valid one. Past the gate, programs rank by improvement over the incumbent on hidden folds. The reference promotion rule keeps the existing incumbent on a tie. Novelty never pays for losing: any novelty bonus (Section 5) must respect the order. For a stillborn program $\tilde{w}$, an also-ran w^- and a win w^+ on board p,

$$r(p,\tilde{w}) < r(p,w^-) < r(p,w^+).$$

The opponent is a ratcheting incumbent. A program must strictly improve under the same evaluation protocol to displace a board's incumbent. The reference design gives a retrained writer a separate proposed admission test (Section 6.5). The two rules define its self-play: a proposer faces incumbents retained from earlier play. AlphaGo Zero likewise separated network admission from move selection [101]. These are reference rules, not a record of writer improvement. The later loop in Section 6.8 updates only a ranking adapter; its default admission gate can retain equal utility. That record does not establish retraining or admission of the neural writer.

Subtracting the incumbent's score does not change which move is best. Promotion raises the bar, turns yesterday's winners into today's also-rans and shifts which problems are worth playing. The reference learning targets use each program's rank within its scored list (Sections 6.4 and 6.5).

4.5 One game, many boards

Casting algorithm discovery as a game is not new, but earlier games wrote one family of problems into their rules, as AlphaTensor did for matrix multiplication and AlphaDev for sorting [33, 66]. AlphaZero used one algorithm for three games and trained a separate network for each [102]. Here the problem is an input on the board, so one player can, in principle, learn many boards.

The historical reference checkpoint and the comparison protocol here are scoped to tabular regression. Binary and multiclass classification, ranking, forecasting and relational prediction remain prospective boards, each needing a qualified encoding, program language, metric and validation protocol.

Beyond machine learning, examples of referees include sorting tests [66], evaluators for mathematical constructions [91] and accelerator-kernel timers [75]. These motivate extensions of the recipe, rather than results of this paper (Section 8).

Key idea

A reliable referee makes a discipline a candidate for this game; transfer to that discipline still needs evidence.

4.6 The analogy's limits

Four limits qualify the analogy to board games. **The referee is noisy:** picking a winner by a noisy validation score biases that score upward [17]. Strict numerical improvement alone does not quantify confidence; independent evaluation and an uncertainty protocol are needed to interpret it. **Attempts consume budget:** fitting, validation and failed execution all incur costs; invalid programs may stop before fitting (Section 5). **The referee can be gamed:** the evaluation harness must resist exploits [114]. Section 6.6 specifies the isolation and parity checks required here. **No algorithm wins everywhere:** averaged uniformly over all problems, no learner has lower error off the training set than another, ours included [118].

Real problems are structured, though, and a player that reads the board can choose its bias board by board. Section 7 reserves the tests of that hypothesis against fixed frontier-designed regressors, with paired outcomes beside Elo summaries [32]. Non-transitive comparisons are one reason not to reduce the evidence to a single rating [4]. A worked game, chosen by a rule fixed before inspection, is to be reported with the results (Section 7.6).

The game defines an attempt and a verdict. Section 5 accounts for the work required to obtain a verified improvement, including invalid programs and valid losses.

5 The Price of Play

The bill for a verified improvement includes the winning attempt and every earlier failure. It covers encoding, input tokens, generation, repairs, referee execution, updates and operating costs. The planning ratios below organize that bill; a full-run price requires the measurements in Sections 5.6 and 7.

5.1 The bill in one line

For independent attempts with fixed win probability and cost distribution, expected cost per win is mean cost per attempt divided by that probability (Equation (1)):

$$C_{\text{win}} = \frac{C_{\text{attempt}}}{q}. \tag{2}$$

Here C_{attempt} is the mean incurred cost per candidate attempt, including failures, and q is the probability that an attempt beats the incumbent. A repair that emits a revised candidate is another attempt; charge its work once.

Write $q = q_{\text{run}}\, q_{\text{beat}}$, where q_{run} is the share of protocol-valid attempts and q_{beat} the win rate among them. Report both with the overall win rate; q_{beat} is undefined if no attempt is valid. Valid ties and losses still cost money. Grammar constraints or selection using revealed outcomes can alter the validity rate without weight updates (Sections 6.3 and 6.8). These rates alone do not identify a learned improvement (Section 7.5).

For an observed batch, cost per verified win is the total ledger divided by verified wins, including update and operating costs where applicable. This accounting does not require independent attempts. Adaptive search still needs empirical cost and attempt curves; a batch with no verified win has no finite observed cost per win.

Training and deployment need separate ledgers. Training pays for every attempt, including failures. The proposed deployment protocol pays for encoding, one proposal and its fit, with no repair. Fixed-weight agent loops pay for evaluation during task-specific search. Training a proposer may shift some of that cost to earlier tasks; the crossover still needs measurement (Section 7).

5.2 Wasted moves

The ledger distinguishes stillborn attempts from valid losses. A stillborn attempt fails to produce a usable program under the declared protocol: generation is incomplete, parsing or type-checking fails, execution exceeds its limits or crashes, or outputs fail predeclared validity checks. An also-ran is valid but does not beat the incumbent. The pilot funnel planned for Section 7 (Figure 13, panel a) traces those stages and the compute spent before each failed attempt stops.

If 90% of attempts fail to improve the incumbent, the independent-try model requires ten attempts per improvement in expectation. Although a typed language can exclude invalid structures before fitting (Section 4), runtime, resource and output failures can still occur after fitting has begun, so the pilot must measure whether valid also-rans dominate. The matched writer comparison in Section 7.2 holds the deployment budget, incumbent and referee fixed.

5.3 The correction tax and the reasoning dial

Generating a token costs about 2 FLOPs per active parameter, and training on one about 6 [48]. Here and in Section 5.6, we consider language-model writers for regression and the agent comparators of Sections 7.3 and 7.4. The compact reference network and its generation limit are specified in Section 6.

Small models are cheap per token and can be dear per win. For generation alone, compute per token falls about 190-fold from a hypothetical dense 1.5T writer to an 8B one. With equal token counts and compute unit costs, the smaller writer can tolerate a win rate about 190 times lower before that component's cost per win matches. Section 5.6 separates active compute from resident model memory.

If the smaller writer needs three times the tokens per attempt, the generation-only ratio falls to about 60. Repair turns also reread a growing context. Encoding and referee costs change the full-cost ratio; a threshold for the complete bill in Equation (2) requires counting every component.

Reasoning is the second dial. Reasoning tokens may cut stillborn moves, but each is billed as output. The planned pilot (Figure 13, panel b) compares cost per win over writer size and reasoning effort. A U-shaped curve is the hypothesis; the measurement decides where, and whether, the minimum sits.

5.4 Rewarding useful hallucination

A familiar solution can be a useful prior. Matching the incumbent's score earns a tie; promotion requires improvement under the same protocol. A *useful hallucination* here is an unfamiliar proposal whose value the referee verifies (Box 3). Possible design options for exploration include temperature and entropy schedules, a curriculum past known solutions and quality-diversity archives [62, 72]; any novelty bonus must preserve the reward ordering in Section 4.4.

The reference training routes combine policy draws with grammar-based fallback and random exploration to seed verified search (Section 6.5). An exploration study must declare those controls and their budget shares, including unsuccessful attempts; Section 7 reserves the quality–cost comparison.

Box 3: What a referee verifies

An unfamiliar program is a conjecture; the referee checks whether it runs and improves the declared score. Verification establishes that bounded result. A claim of discovery also needs comparison with known methods (Section 8). The Go example in Section 2.4 illustrates this distinction. The referee's reliability and execution cost must both be established for the task (Section 6.6).

5.5 How many games?

Attempts, training experience and test precision answer different questions. **Tries to find one improvement:** $1/q$ on average (Equation (1)). For independent tries with fixed $q = 1\%$, 69 attempts reach even odds and 299 reach a 95% probability of at least one win in that model. Report empirical attempt curves for adaptive or correlated proposals.

Experience to train a transferable player: AlphaGo Zero generated 4.9 million self-play games in about three days [101], and AlphaZero played 44 million games of chess

in about nine hours [102]. An illustrative grid of 10,000 problems, 100 rounds and 8 attempts per problem per round gives 8 million attempts, the same order of magnitude. That grid counts raw attempts: transfer still requires evidence across independent problem families, and pilot learning curves support claims only within their measured range.

Precision to plan the test: illustratively, 150 independent units at a win rate near 60% give an approximate 95% interval of ± 8 points. The actual study must compute uncertainty over independent families, rather than treating every dataset as an independent observation.

5.6 Pricing evidence still required

Turning the per-token scaling comparisons of Section 5.3 into a hardware bill requires accounting for resident model memory alongside active compute. Mixture-of-experts writers spend compute on active parameters but require memory for all parameters: Kimi K2 has 32B active of 1T, while DeepSeek-V3 has 37B active of 671B [27, 51]. Resident one-byte weights for 1T parameters alone require at least 13 H100 SXM GPUs with 80 GB each [76], and runtime state needs additional capacity. This hardware memory floor is separate from the full run-cost projection discussed next.

A reproducible projection therefore needs a pinned model and hardware configuration, dated prices, measured or justified utilization and an explicit accounting boundary. Those inputs remain unrecorded; a full cost projection remains a measurement task. Pricing the adapter operating clocks (Section 6.8) still requires an attempt ledger.

Key idea

Cost per verified improvement depends on both the cost of an attempt and the probability that it wins.

Under Equation (2)'s fixed-probability model, doubling q halves expected cost per win only if mean attempt cost stays fixed. Measuring that tradeoff requires a common task mix, incumbent and verification rule. In the reference protocol, referee-scored attempts supply training targets (Section 6); the adapter record is a separate operating observation.

6 EurekaZero: Learning Taste

Search-scored lists supply the reference training targets for a writer and a separate ranking map. The proposed deployment protocol then freezes the writer and commits one proposal. The adapter record in Section 6.8 is a separate loop, with its own cost ledger (Section 5) and evaluation.

6.1 The player in one picture

A frozen encoder reads the problem board (Figure 4); the reference writer emits a complete program under the contract in Box 1, as AlphaGo's policy proposed a move [100]. The separate ranking map enters training search through the routes described in Section 6.4.

6.2 Reading the board

The reference regression encoder turns the board of Section 4 into 2,048 numbers, $z_p = \phi_{\mathrm{P}}(p)$, in three blocks. The first summarizes a pretrained row encoder's embeddings across rows, independent of row order (512 values). The second projects the hidden state of a pretrained third-party tabular regressor that reads the training rows and labels (1,024; third-party provenance pending; restricted; redistribution prohibited). The third summarizes two 32-element windows of training targets scaled to $[0,1]$ (512 values; constant targets map to zero).

Preparation caps the combined tabular splits at 50,000 rows. Encoder inputs remain training-side only: the regressor block samples at most 768 context rows and 256 query rows from those training rows. Frozen meta-training cohort moments standardize each component block before concatenation and elementwise clipping to $[-32, 32]$.

The reference encoder is pinned by hash and stays fixed during writer training. Encoding is bounded at 30 s per problem. A context with any non-finite value is refused rather than imputed (Appendix C). The input contract permits training-side task evidence only; final held-out labels and their derived statistics must stay outside the input closure. A context's recorded declaration that no final target was used establishes neither input closure nor numerical qualification. CPU contexts require native replay and bounded-execution receipts before accepted-policy use.

Whether problems with similar best programs sit together in that space requires a separate representation study; Section 7.5 reserves that evaluation test.

6.3 Describing the move

A move is one program in the typed language of Section 4, with a 401-token training inventory listing 31 transforms and 22 prediction heads; the product runtime restricts executable heads. The writer emits a flat operator-and-slot token stream with a fixed beginning, engine and regression-task header, rather than parenthesis tokens or the `pipe` wrapper. Its nine-phase grammar mask permits up to six transform stages and three expression stages; each expression tree is bounded at depth four and 15 nodes [24, 56]. After the end token, detokenization renders a parenthesized program that must parse before entering the pool. The mask has no remaining-token-budget check: a draw without an end token at the 128-token horizon, including the fixed header, is incomplete and discarded. Grammar legality does not establish successful execution or useful quality. The proposal pool removes exact decoded-string duplicates and records their SHA-256. The executor also records canonical recipe hashes. Probe-based behavior matching is a separate audit, not part of pool deduplication. Without probe evidence, this behavior check reports novelty as unknown. Readable Python and Rust exports are a reference requirement. Legacy recipe emitters reject the synthesized input stage; the separate fitted-state path is allowlisted. Reported winners still require the parity check in Section 6.6.

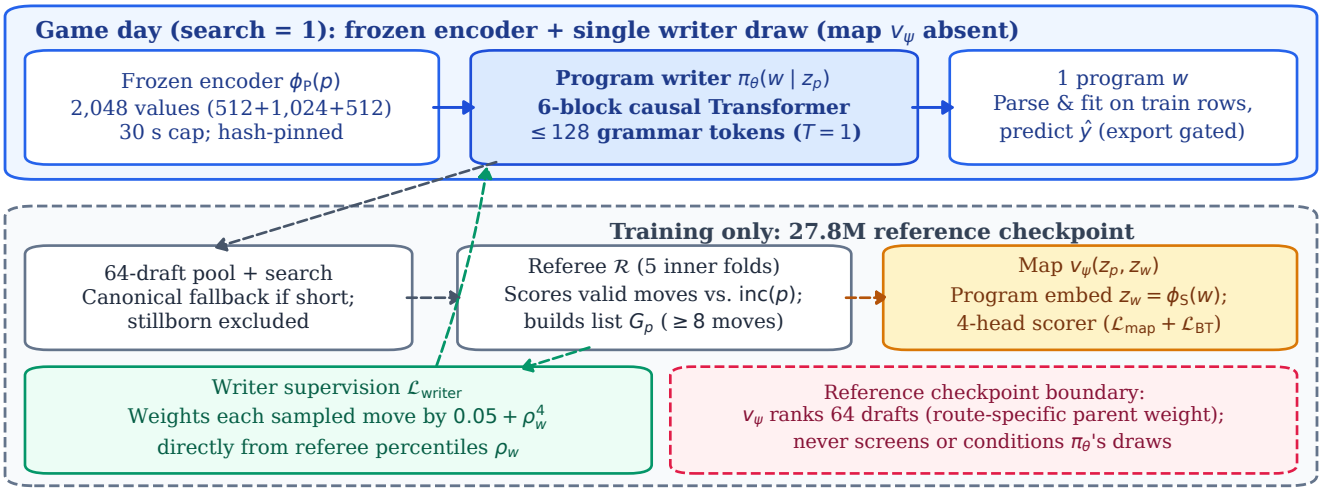


Figure 4: **Reference training updates the writer and map; game day uses the writer alone.** Top: game day, the whole of search = 1. Dashed box: proposed training. Search extends the writer's drafts; referee-scored lists supply objectives for the writer (θ) and map (ψ). Stillborn programs enter no list.

6.4 One map for problems and programs

The reference map ranks programs within a problem by decreasing logit. It reads a program as $z_w = \phi_S(w)$, the layer-normalized mean of non-padding token embeddings, including `BOS`, the fixed engine and task header, and `EOS`. Pooling discards token order, limiting its ability to distinguish compositions built from the same tokens.

The map's own two-layer context network projects the 2,048-value z_p to $r_z \in \mathbb{R}^{512}$. It joins three 512-value blocks: z_w as is, $z_w \odot \tanh(W_\gamma r_z + b_\gamma)$ and $|z_w - (W_{\text{proj}} r_z + b_{\text{proj}})|$. Four two-layer heads receive this same 1,536-value interaction vector; their mean gives the ranking logit $v_\psi(z_p, z_w)$. The reference ranking losses supervise that mean jointly, without separate targets or bootstrap samples. Training applies dropout of 0.05 in each ranking head. The runtime head instead reads $[r_z, z_w]$, while the program-only control reads z_w.

The score is not a calibrated error estimate. Comparing its magnitude across problems or reading it as a probability of a verified win requires separate evidence. The referee still establishes validity and measures performance.

The map's training objective uses scored lists G_p for problem p; $\rho_w \in [0,1]$ is a program's percentile, 1 for the best. Native training requires at least eight scored programs in a source list and samples eight per comparison: its champion and seven quantile-stratified comparators. The lower-level listwise loss accepts groups of at least three with finite percentile targets (Appendix C). The list-level form trains the map's softmax toward a target concentrated on the best [16]:

$$\mathcal{L}_{\text{map}}(p) = -\sum_{w \in G_p} \frac{e^{\rho_w/\tau}}{\sum_{u \in G_p} e^{\rho_u/\tau}} \log \frac{e^{v_\psi(z_p, z_w)}}{\sum_{u \in G_p} e^{v_\psi(z_p, z_u)}}, \quad (3)$$

with native target temperature $\tau = 0.25$. Temperature scales the target percentiles; the mean ranking logits enter the prediction softmax unscaled. With a unique best program, the low-temperature limit concentrates the target on that program. This is a listwise ranking objective over programs scored on the same problem; the analogy to contrastive learning [88, 111] does not establish CLIP-style alignment across problems and programs. A Bradley–Terry term adds pairwise comparisons on the same lists [12]. Stillborn moves have no score, so this ranking objective covers only programs that run.

The program-only head controls for ranking without problem context (Section 7.5). Ranking valid scored programs alone does not test the validity rate q_{run} in Section 5.1.

Key idea

The reference game-day protocol omits the ranking map. Matched first proposals therefore test whether training improved the program writer.

In the reference native implementation, the writer and map use separate context networks and token embeddings. The map ranks drafts without dropping them or screening the writer. One native GP route matches map-derived weights by program identity to ranked survivors for parent selection. The bounded search driver instead cycles among up to four parents chosen by measured loss. These are different influence paths, not evidence of trained-map lift; any evaluation must bind the exact route that it used.

Only the 64 neural or fallback seeds receive normalized rank weights; the 16 tail seeds enter unscored. Unscored survivors share the lowest scored survivor weight while scored survivors remain. A cut identifies a fixed dataset split and evaluation case. Native assembly takes up to ten same-cut teachers, then cross-cut teachers up to a 16-seed tail. Random fallbacks fill to ten slots if needed; random exploration fills the remaining slots. The standalone seed writer instead uses 16 random programs. Teacher-first execution prioritizes retained cut teachers; Algorithm 1 leaves the tail route-dependent. Search-parent weights and writer-loss weights act at different stages; the latter use referee rank. Whether map-guided targets improve

later proposals requires the matched ablation in Section 7.5. The historical diagnostics in Figure 34 do not supply that controlled comparison.

In the proposed offline loop, referee-scored search outcomes supply training targets.

6.5 The self-play loop

The native proposal sampler seeks 64 distinct parseable programs at $T = 1$, scheduling at most eight batches of 128 neural draws (1,024 total). To fill a shortfall, it tries up to 10,000 seeded random canonical pipelines and projects them through the grammar. It refuses an incomplete candidate pool. These are training-search budgets, separate from the single-draw game-day protocol in Section 6.7.

The reference rule is expert iteration [1]. The writer term weights sampled programs from G_p by referee rank; it joins the map and auxiliary terms in the full combined training objective (Appendix C):

$$\mathscr{L}_{\text{writer}}(p) = -\sum_{w \in G_p} \left(0.05 + \rho_w^4\right) \log \pi_\theta(w \mid z_p), \tag{4}$$

$$\mathscr{L} = \mathscr{L}_{\text{writer}} + 0.1\,\mathscr{L}_{\text{map}} + 0.25\,\mathscr{L}_{\text{BT}} + 0.1\,\mathscr{L}_{\text{runtime}} + 0.05\,\mathscr{L}_{\text{prog}}. \tag{5}$$

The writer loss in Equation (4) is schematic: it displays the referee rank weights. The native implementation also smooths grammar-legal targets, averages token losses within each program and normalizes by the total sum of program weights (Appendix C).

Here $\mathscr{L}_{\text{BT}}$ is a Bradley–Terry loss on the same lists, $\mathscr{L}_{\text{runtime}}$ a Huber loss between $\ln(1+s)$ and $\ln(1+\text{softplus}(h))$, where s is nonnegative measured seconds and h is the raw runtime-head output, and $\mathscr{L}_{\text{prog}}$ the Bradley–Terry loss of the program-only control head. A median-ranked program carries about a tenth of the top-ranked program's weight in the writer loss. Its referee score also contributes to the map's ranking targets.

The reference procedure uses two ratchets (Section 4.4). A program displaces a board's incumbent only by strictly beating it. The proposed reference admission test calls for 16 held-out cuts with three seeds each. A challenger may replace the serving writer only if its mean log error ratio favors it. It must also win at least as many cuts as it loses and fail no more often. The admission cases are fixed before the test. For each cut, the rule takes the natural log of paired best-error ratios (challenger divided by serving) and averages over matched seeds; it then averages across cuts. Negative values favor the challenger. This admission comparison is separate from the single-program comparison in Section 7.2.

The historical reference. The reference model combines a decoder-only writer with separate ranking and auxiliary heads. Appendix C records the architecture, registered capacity, token allocation and training corpus in one inventory.

These specifications do not supply the checkpoint's acceptance evidence or qualify its runtime. Both records remain inputs to the evaluation contract (Section 7.1). The historical reference checkpoint is scoped to tabular regression. Its native training entry point enforces a dispatch hold. Numerical parity of the CPU float32 Candle port with the native CUDA BF16 tch runtime remains unqualified; random streams and uniform-token fallback differ.

6.6 Keeping the referee honest

Reliable training targets require a referee that resists exploitation. In February 2025 Sakana AI walked back claims after its AI CUDA Engineer exploited the evaluation code [116]. In June 2025 METR reported frontier models patching their scorers [114], and 16% of 1,968 agent-benchmark tasks can be passed without being solved [123].

The evaluation protocol must withstand such attacks. Selection, promotion and final scoring must use separate data (inner folds, held-out cuts, a final fold scored once), with sealed meta-test families outside the learner's inputs. The proposed comparisons also require sandboxed execution using seccomp and Landlock, with memory, swap and time limits. A planted label leak and a sealed canary must both be caught. Every reported winner must be re-run from its emitted code, with native and emitted predictions agreeing within $10^{-9}\max(1, |\hat{y}|)$.

One earlier leaderboard entry for our search selected each dataset's best test-scored outcome from a ledger of 63,108 evaluations; we exclude it as test selection (Appendix B).

6.7 Game day: search = 1

The proposed game-day protocol fixes the components and seed before one regression draw, retaining failures and all incurred costs (Algorithm 2 and Box 1). Section 7.2 compares that draw with a frozen reference under matched conditions.

Report any later search separately, as Silver et al. [101] reported AlphaGo Zero's raw network apart from its lookahead search (Section 6.8).

6.8 A separate adapter admission record

Unlike the reference self-play loop in Section 6.5, the operating record below tracks candidate selection and admission for a cost-ranking adapter; logged adapter utility is a scalar with larger values treated as better.

Typed program edits, cross-family retrieval and grammar synthesis supply candidates in the inspected implementation. The inspected loop freezes every network parameter, then enables updates only for non-anchor parameters of the cost-ranking adapter. This loop has no neural program writer. Its pool-utility calculation uses the recorded scores and gate outcomes of the first four candidates as revealed history, then selects the lowest predicted cost from the pool, including those entries. Training puts the lowest recorded scores first in its slates; utility evaluation uses stored pool order. The calls can therefore reveal different examples. For new proposals, the caller uses predicted costs and an exploration setting to select programs for exact evaluation.

At the adapter interface, the caller supplies modality tokens and their mask. It separately zeros the 584-entry context-feature slot and both prior inputs, and supplies a fixed scale

Algorithm 1 The reference self-play loop (training).

Require: training problems; serving writer π_θ and map v_ψ; incumbents $\mathrm{inc}(p)$

for each round **do**
  for each sampled training problem p **do**
    $z_p \leftarrow \phi_{\mathrm{P}}(p)$; keep the first 64 distinct valid draws from $\pi_\theta(\cdot \mid z_p)$ ▷ $T = 1$; bounded fallback if short
    verified search, seeded with the 64 drafts and 16 tail seeds (teachers/fallbacks and random exploration), proposes more programs ▷ the teacher
    $\mathscr{R}$ fits and scores each distinct program on 5 inner folds ▷ stillborn: no score
    if a program strictly beats $\mathrm{inc}(p)$ **then** it becomes $\mathrm{inc}(p)$ ▷ board ratchet
    add every scored program to the list G_p ▷ winners and also-rans
  train a challenger (θ', ψ') on the lists
  if it beats π_θ on held-out cuts **then** $(\theta, \psi) \leftarrow (\theta', \psi')$ ▷ proposed player ratchet

Algorithm 2 Proposed game-day protocol (search = 1).

Require: new regression problem p with training rows and metric; frozen ϕ_{P} and π_θ

$z_p \leftarrow \phi_{\mathrm{P}}(p)$ ▷ read the board
$w \sim \pi_\theta(\cdot \mid z_p)$, one draw at $T = 1$ ▷ one move; no retry
$f \leftarrow \textsc{Fit}(w,$ training rows$)$ ▷ as any algorithm is fit
$\hat{y} \leftarrow f($new rows$)$ ▷ a stillborn w scores as a loss
return w, $\hat{y}$

constant of 0.4208; zero context features do not mean absent modality tokens. Its objective combines regression to recorded costs and listwise matching; failed gates receive target cost $+2.0$. Reporting averages negative selected scores rather than cost predictions, with utility -2.0 for failed gates. The full-cohort route disables development-anchor margin projection; smaller-cohort runs can enable it. These two operating routes require separate verification receipts.

These selection records alone do not establish a gain from optimizer updates or from the separate reference map. The limits of component diagnostics are discussed in Section 7.7. Testing learning requires matched trained and untrained controls with the same pool and revealed history, reporting invalid selections separately from valid-program quality (Section 7.5).

Two admission decisions. Program candidates use two training-side folds; the historical reference specifies five. Evaluation logging does not imply training-pool admission. Ordinary admission excludes failed gates and non-improving candidates: lower scores must beat the context incumbent by more than 10^{-6} (Figure 26). Historical programs retain a separate admission path.

The adapter accepts ties by default, so a recorded best-round identity can change without raising utility. Optional strict admission requires utility above the retained best plus an explicit nonnegative margin. Rejection restores the retained model and optimizer states. GP search continues, and the completed-round clock includes rejected adapter work.

Scope of the telemetry. All records here and in Appendix D are adaptive training-side observations. Completed-round clocks include all work and qualification waits; clock sums omit gaps between rounds. Program-incumbent events and adapter admissions have different denominators; neither establishes independent first-proposal improvement or deployment throughput. The aggregate records do not yet bind an exact loop source, adapter, evaluation population or normalization, so cross-model comparisons and transfer remain unqualified. Retained windows are separate; unobserved gaps are not filled or joined into a learning curve. Independent comparisons belong to Section 7.

[TBD: Bind the retained snapshot to the exact loop source, adapter, candidate pools, revealed-history inputs and utility normalization.]

The retained admission ledger. Table 10 is the canonical chronology and count table. Figures 5 and 6 show its trajectory and pre-decision margins.

Figures 7 and 8 add companion operating-rate views for separate retained windows.

A later admission window. Figure 9 shows the latest 200 complete admission decisions at a later capture (all rejections), plotting every candidate and post-decision retained utility on reused development contexts under the same source-binding limits described above.

Pool-insertion cadence. Figure 10 measures the timestamp gaps between logged training-pool insertions (distinct from adapter admissions), including intervening work and waits rather than single-proposal runtime.

In Appendix D.9, Figure 22 tracks the recorded best and Figure 23 counts pool insertions and adapter rejections (Appendix D.18 gives the program clock; Appendix D organizes the full record).

Aligning the committed event histories. Figure 24 counts pool insertions and adapter admissions over a common round window. Their event meanings remain distinct; this alignment does not join evaluation populations or establish a learning gain (Appendix D.10). Figure 25 resolves that insertion history into fixed round windows, separating local event frequency from the cumulative count (Appendix D.11).

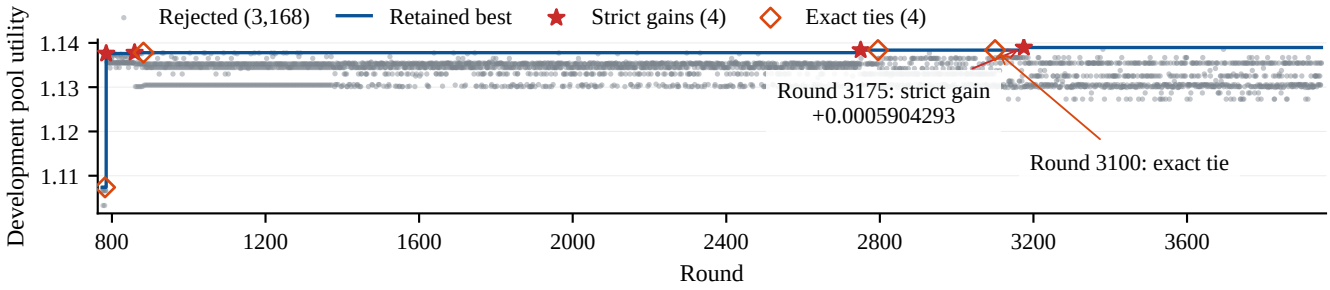


Figure 5: **The adapter admits eight candidates across the captured prefix.** In Table 10, blue is the post-decision retained best; grey marks rejected candidates, red stars strict gains and hollow diamonds exact ties. Decisions use unrounded logged values. Scope: Section 6.8.

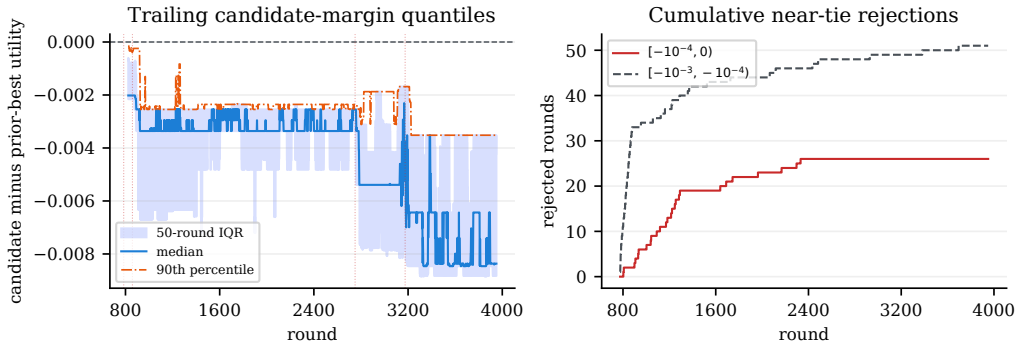


Figure 6: **Most rejected candidates fall below the retained best.** Left: trailing 50-round median, interquartile range and 90th percentile of candidate minus pre-decision retained-best utility, including admitted candidates. Quantiles begin at the first complete 50-round window; dotted vertical lines mark strict-gain rounds. Right: cumulative rejected rows in two exclusive margin bands. Both panels use the full prefix in Table 10; no unobserved rounds are inferred. Scope: Section 6.8.

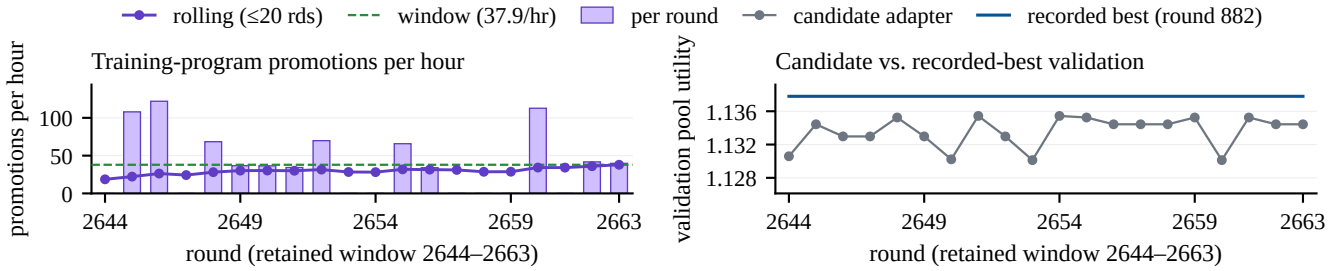


Figure 7: **Training-program promotions continue while the retained adapter holds flat.** Left: logged rolling and per-round strict training-program promotions per hour. Right: candidate and recorded-best adapter validation pool utility. Both panels cover the 20 consecutive completed rounds 2644–2663 (21 strict promotions over 1,994.54 s of logged wall time, or 37.90 promotions per hour, with zero adapter admissions). Scope: Section 6.8.

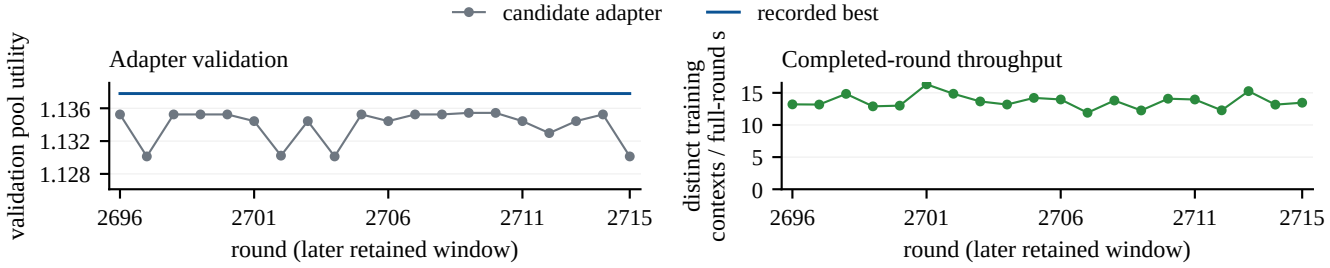


Figure 8: **Whole-round throughput varies while retained adapter utility holds flat.** Left: candidate adapter validation utility and the recorded best. Right: distinct training contexts divided by the full wall time of each completed round (contexts per second). The rates include all work and waits inside that interval. Both panels cover the same 20 completed summaries, rounds 2696–2715. Scope: Section 6.8.

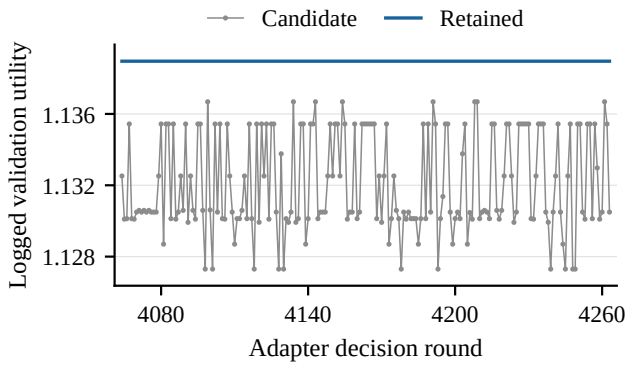


Figure 9: **Candidate utility varies while the retained adapter holds.** All 200 consecutive admission decisions in rounds 4064–4263, captured at 10:19 UTC on 3 November 2026. Grey: candidate validation-pool utility; blue: the logged post-decision retained utility (dimensionless, larger preferred). Every point maps to a journal line; no smoothing or cumulative maximum is applied. The window has zero admissions. This adaptive engineering telemetry does not measure independent first-proposal performance or a learning gain. Scope: Section 6.8.

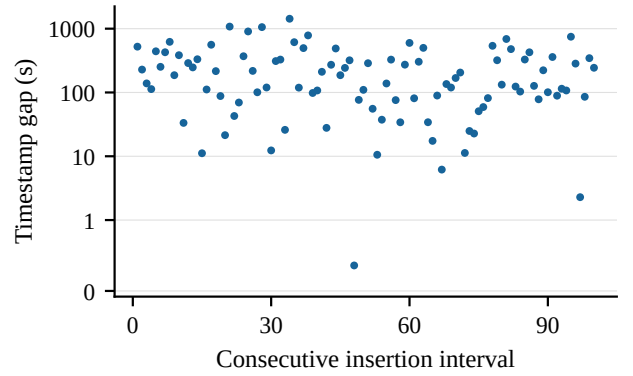


Figure 10: **Pool-insertion timestamps are unevenly spaced.** The latest 101 complete insertion records at 10:51 UTC on 3 November 2026 span rounds 4051–4280 and yield all 100 adjacent timestamp gaps. Each point maps to two consecutive journal records; all gaps are retained without a duration cutoff. The vertical axis is symmetric-log with a linear region within one second. These engineering timestamps include work and waits; they do not measure proposal runtime, adapter admission or independent learning gains. Scope: Section 6.8.

7 Results

With the single-candidate budget fixed, the tests below ask whether training improves a first proposal on an unseen family. The development records in Section 6.8 concern program search and adapter admission; their clock and decisions do not establish deployment performance. Table 5 separates the training, fixed-method, search and map comparisons from the failure analyses. All first-proposal evaluation outcomes remain pending. **[TBD: Add independent evaluation evidence before making a performance claim.] [TBD: Supply a source-bound benchmark before claiming deployment throughput.]**

7.1 Evaluation contract

Freeze the writer, permitted task inputs, output support, decoding rule and budgets before opening final outcomes. Pin dataset versions and content hashes, row splits and family assignments. Separate unseen problem families from training and development, and report which task types were actually tested.

Report known training and pretraining exposure for the writer, encoders and learned comparators, with unresolved provenance marked explicitly. Bound unseen-family claims to the exclusions actually established.

Each EurekaZero submission follows Algorithm 2, counting failed generations and executions in the overall win rate. Also report validity and wins among valid attempts; neither rate alone establishes learning. Pair outcomes and uncertainty by family, with encoding, generation, fitting and prediction costs. Disclose internal model or hyperparameter selection within the emitted procedure. **[TBD: Document the final problem inventory, dataset identities, row splits, family assignments and exclusions, component provenance, evaluator, checkpoint, seeds, task metrics and failure accounting.]**

7.2 The effect of training

Use a frozen reference writer as the control. Predeclare its checkpoint and the evaluated training intervention before opening final outcomes. Hold the encoder, program grammar and output support, permitted task inputs, decoding rules and fitting protocol, referee, incumbent and resource budgets constant; vary the writer checkpoint. Changes to the support, encoder or evaluator belong in separate conditions.

Pair the first draws under predeclared seeds and report all of them, with no selection of the best seed or candidate. Report paired family-level outcomes, failures and costs alongside source-bound evidence of optimizer updates and parameter changes. **[TBD: Supply the matched first-proposal comparison and training-update evidence before claiming transfer.]**

7.3 One proposal against fixed methods

[TBD: Identify Jepson's supplied frozen comparators, their provenance and eligibility, and the matched outcomes and uncertainty for Table 6 and Figure 11.]

7.4 The cost of more attempts

Compare the single proposal with agents allowed successive attempts and development feedback. Keep final outcomes unavailable to those agents. Count every candidate attempt, including incomplete or invalid outputs and failed executions; a repair that emits a revised candidate is another attempt. Charge all task-time computation.

Plot performance against both attempts and cost (Figure 12); if an agent does not reach a reference performance within its

Table 5: **Evaluation plan at a glance.** Each test is specified in the subsection named in the first column. No outcome is reported yet; the contract in Section 7.1 fixes inputs, budgets and splits before outcomes are opened.

Test	Question	Comparison or control	Reported in	Status
The effect of training (Section 7.2)	Does training improve the first proposal on unseen families?	A frozen reference writer under the same budget, paired first draws	Text of Section 7.2	Pending
One proposal against fixed methods (Section 7.3)	Does one task-conditioned program compete with methods that run unchanged across tasks?	Frozen algorithms whose receipts record the designing model	Table 6 and Figure 11	Pending
The cost of more attempts (Section 7.4)	What do successive attempts buy, per attempt and per unit of cost?	Agents allowed successive attempts; the pilot cost funnel	Figures 12 and 13	Pending
What the map contributes (Section 7.5)	Does the map read the problem, and does it change the writer?	The program-only head; a matched causal ablation	Table 7	Pending
Failure cases and a worked example (Section 7.6)	How do attempts fail, and what does one game look like?	Terminal outcome classes; a rule-selected example	Table 8	Pending

Table 6: **Planned paired comparisons.** No outcomes are yet supplied. Report ties, failures and excluded cases alongside wins.

Comparator	Outcome and uncertainty
[TBD: first frozen comparator identity]	**[TBD: paired evaluation]**
[TBD: second frozen comparator identity]	**[TBD: paired evaluation]**

[TBD: Leaderboard: supply the evaluated methods, paired outcomes, rating protocol and uncertainty.]

Figure 11: **Leaderboard pending.** Elo is a summary of a specified comparison population, not a substitute for paired task results.

budget, report that limit. **[TBD: Supply attempt curves, matched budgets, repeated seeds and uncertainty.]** Pilot cost measurements from Sections 5.2 and 5.3 are planned for Figure 13.

7.5 What the map contributes

Test whether nearby problem encodings share useful programs, using data disjoint from the data that fit the map. Compare ranking agreement with the program-only head, so that a global preference for common programs can be distinguished from reading the problem. **[TBD: Supply problem-similarity and program-only-control evaluations.]**

Agreement with the referee tests ranking, not improvement of the writer (Table 7). The reference map neither conditions nor screens a writer draw (Section 6.4); its accuracy on valid scored lists cannot establish a reduction in invalid proposals or an overall win-rate gain.

For a causal test, bind the parent-selection route and evaluated checkpoint. Vary the map's influence while holding the initial writer, candidate-generation rules, training budget and deployment protocol fixed. Measure the resulting writers' paired first proposals, including validity, wins and incurred costs. **[TBD: Identify an implemented influence path or limit the study to ranking.]**

[TBD: Search curve: performance versus attempts and compute, including failed attempts and unreached targets.]

Figure 12: **Deployment search comparison pending.** The single-candidate condition fixes a budget; its measured quality remains to be established.

[TBD: Pilot cost measurements: supply per-stage failure shares and cost per verified win.]

Figure 13: **Pilot cost measurements pending.** Planned panels: (a) the share of development proposals that run, score, beat the incumbent and survive re-verification, with the compute spent at each stage; (b) cost per verified win by writer size and reasoning effort. **[TBD: Supply pilot data, seeds and family-level uncertainty; identify settings with no verified win without assigning a finite cost per win.]**

Table 7: **Map evidence still required.** Ranking quality and a change in first-proposal quality are separate outcomes.

Question	Required evidence
Does the map rank candidates?	**[TBD: held-out ranking study]**
Does it improve the writer?	**[TBD: matched causal ablation]**
Does it reduce execution failures?	**[TBD: failure-class comparison]**

7.6 Failure cases and a worked example

Describe unsuccessful proposals and coverage limits as well as wins. Assign each attempt one terminal status: incomplete generation, parse or type-checking refusal, execution or resource failure, invalid output under declared checks, and valid

ties, losses and wins. Charge every attempted candidate its incurred cost. Unresolved outcomes retain an explicit unknown status; fix their treatment and the validity rules before opening final outcomes. **[TBD: Supply attempt inventories and costs by outcome class, identifying unresolved cases.]**

Select the worked example by a rule fixed before inspecting outcomes. Report its emitted program, fitting procedure and comparison outcome. If the selected category is empty, report that fact. **[TBD: Supply the example selection rule, actual program and verified outcomes, with no replacement of failed attempts.]** Table 8 lists the record for the selected game. The attempt permits no repair or second try, and its sealed verdict is reported without updating the writer.

7.7 Earlier development diagnostics

Closed-out development studies evaluate a frozen reference adapter and its modifications, rather than the writer's independent first proposals. The first three studies use 1,219 development-training contexts from 205 families and three seeds with different five-way family partitions. Prior fitting and gradient-clipping continuations exclude evaluation and selection families. Test-time adaptation uses only the four revealed candidates in each context and resets its weights after scoring. Fits or settings were locked before outer-fold evaluation. Contexts are averaged within families; families receive equal weight. Utility-gain intervals use 10,000 paired-family bootstrap draws shared across seeds and summarize the median seed gain. Reported intervals are 95%; p values are one-sided, with one primary contrast per study. These studies reuse a development population; they are not independent replications.

Calibrated blend. A calibrated tree prior receives weight 0.25 in a cost-space blend with the reference adapter. Its primary endpoint is the maximum utility among four revealed and four lowest-cost unrevealed candidates, rather than a single native choice. The gain is $+0.0137$ (95% interval $[+0.0056, +0.0259]$; $p = 0.0005$ against a $+0.001$ margin). Elo is 1523.1 $[1513.5, 1532.4]$, relative to the reference at 1500. Family harm below -0.01 is 7.3% (15/205). The result is a **candidate**, with no adoption and zero adapter optimizer updates; the tree prior was fitted.

Per-context clipping. Clipping each context's gradient before averaging, with 40 updates per trajectory, gives utility gain -0.02510 ($[-0.05128, +0.00009]$; $p = 0.9743$; 0/3 positive seeds). Elo is 1485.63 (lift -14.37, 95% interval $[-24.34, -4.38]$). Family harm below zero is 46.83%, and the treatment does not strictly beat its shuffled-factor placebo: **NO_ADVANCE**.

Revealed-only adaptation. Selection on a separate panel chose one SGD step at learning rate 10^{-5} in all 15/15 cells. At that locked setting, adaptation changes scores in 1,219/1,219 contexts but changes no selected program. The largest grid budgets flip only 1–4 of 1,219 choices. Utility gain is 0.000000, with interval $[0.000000, 0.000000]$. Elo is 1500.00, with interval $[1500.00, 1500.00]$. The verdict is **NO_ADVANCE (inert treatment)** within the tested one-to-five-step, 10^{-5}–10^{-4} learning-rate grid; it does not exclude effective adaptation at larger budgets.

Table 8: **Worked-game record.** Fields reported for the game selected by the rule above. An empty selected outcome category is reported rather than replaced.

Field	Record
Problem, task and metric	**[TBD: Identify the problem, task and metric.]**
Encoding time	**[TBD: time]**
Emitted output	**[TBD: program or unfinished draft and size]**
Check and fitting outcomes	**[TBD: outcomes and time]**
Hidden-fold score of a valid fit	**[TBD: score]**
Incumbent's score	**[TBD: score]**
If no valid fit	The failure and its incurred cost, in place of the scores

Admission audit. The sole primary attempt is **INCOMPLETE**, with formal verdict **INCONCLUSIVE**: one of 91 choices in 19 development-validation families differs from its archived choice, failing the native parity gate. Post-failure descriptive replay of a stricter admission rule accepts 0/2,418 historical proposals. This replay neither qualifies the checkpoint nor establishes future power or sequential error control.

The single-choice studies use the selected candidate's utility, with invalid choices assigned -2. Their endpoint differs from the blend's best-of-eight endpoint; the reported Elo values are specific to each protocol. Checkpoints, cohort identities, seeds and locks are pinned by the linked reports and their accompanying protocol and metrics records.

None of these closeouts adopts a checkpoint or establishes learning of independent first proposals, deployment throughput or unseen-family transfer. The final comparisons above and their interpretation in Section 8 remain pending.

8 Discussion

Discussion is reserved until the pre-registered evaluation in Section 7 is complete and unblinded. **[TBD: Supply a source-grounded discussion of the completed evaluation from Jepson. Reconcile every conclusion below with those outcomes and their uncertainty.]**

8.1 Learning to select and learning to invent

[TBD: Use Jepson's supplied output and archive comparisons, with program behavior checks, to distinguish selection from useful new composition.]

8.2 Moving judgment across problems

[TBD: Interpret transfer of first-proposal quality across problems using the matched conditions and paired family-level uncertainty supplied by Jepson.]

8.3 The referee defines the boundary

[TBD: Discuss referee validity, contamination, admission versus generalization, and documented failure limits from Jepson's source.]

8.4 A bounded claim about self-improvement

[TBD: State which level of self-improvement is supported by source-bound training changes and independent proposal outcomes supplied by Jepson.]

8.5 Limitations

[TBD: Describe limits on task coverage, the program language, referee reliability and measured cost using Jepson's supplied source.]

9 Conclusion

[TBD: Supply the conclusion and next steps justified by Jepson's results and their limits; reserve stronger claims until their required evidence is supplied.]

A Referee Inventory

A reliable, affordable referee is a condition for the innovation game (Section 4.5). Table 9 inventories the evaluators behind algorithm selection [90], typed and evolutionary search [53, 71, 89], self-play and expert iteration [1, 93, 101], and tabular amortization [34, 44, 73] discussed in Sections 2 and 3.

The inventory identifies the evaluator; family allocation is a separate requirement. In the native audit path, approved datasets are grouped by source identity, declared family or identical training bytes before seeded family ordering assigns the meta-training, meta-validation and meta-test partitions.

This code path does not establish that a corpus or checkpoint satisfies the required partition split.

B Incident Log and Audit Safeguards

Two safeguards bound the comparisons in Section 6.6: exclude test-selected search outcomes, and require final-evaluation receipts for input closure and execution isolation:

- **Excluded test-selected search baseline.** An earlier internal leaderboard entry selected each dataset's best test-scored outcome from 63,108 evaluations in total across 42 datasets. Selection using test-fold scores biases the reported metric upward [17]. The entry is therefore excluded from all comparative claims.
- **Input-closure and canary checks.** Benchmark release requires passing receipts for an inner-fold label-poison test, a sealed canary and a train-only input-closure audit. The audit must exclude held-out targets and derived statistics from both board encoding and candidate generation. Final pre-registration and receipts belong to the primary evaluation record in Section 7.1.

Table 9: **Each domain in the main text relies on an explicit referee.** Each row restates a referee from Sections 3 and 4; the last row states the reference referee for algorithm invention.

Domain	Referee	Systems
Board games	The rules of the game and its final score	AlphaGo Zero, AlphaZero [101, 102]
Machine-learning research	A sealed test set that scores every entry	Netflix Prize, ImageNet [7, 29, 92]
Code	Unit tests; a code executor	Codex, Absolute Zero [19, 122]
Matrix multiplication	The rules of TensorGame	AlphaTensor [33]
Sorting	Sorting tests	AlphaDev [66]
Mathematical constructions	Programmatic evaluators	FunSearch [91]
Accelerator kernels	Timers	AlphaEvolve [75]
Formal proofs	The Lean proof checker	AlphaProof, STP [28, 45]
RNA design	Laboratory tests of designs	EteRNA [60]
Algorithm invention (this paper)	Hidden-fold scores against the incumbent; promotion on strict improvement	EurekaZero (Sections 4.4 and 6)

C Training Details

The historical reference in Section 6 still requires checkpoint qualification. Its encoder, training objective and execution requirements are inventoried below, separately from the adapter record in Section 6.8:

- **Board encoder blocks (Section 6.2).** The frozen 2,048-value context $z_p = \phi_{\mathrm{P}}(p)$ joins three blocks: (i) 512 pooled values from a pretrained row encoder of width 128 across training rows by mean, population standard deviation, and 10th and 90th percentiles; (ii) 1,024 values projected from a 2,048-value third-party tabular regressor state capped at 768 context rows and 256 query rows (the external CPU path first subsamples to at most 1,000 training rows; third-party provenance pending; restricted; redistribution prohibited); and (iii) 512 values from two frozen seed-17 target-summary windows. Each block is normalized before concatenation and clipping to $[-32, 32]$. Non-finite contexts are refused. Encoding records report wall-clock latency from 11.7 s to 20.8 s across eight reference encoding jobs, within the 30 s cap.
- **Comparison construction (Sections 6.4 and 6.5).** Lists G_p require at least eight valid scored programs per problem, with percentile $\rho_w = 1 - \mathrm{rank}/(|G_p| - 1)$ and writer π_θ sequence weight $0.05 + \rho_w^4$. The base native sampler balances families and takes the best program plus one draw from each of seven rank bands of the rest. Fresh-program

replay can replace a comparator to include a newly admitted candidate program.

Pairs stay within each comparison, keyed by corpus context and comparison index. The Python ranking helpers instead group by dataset identifier. The native helper skips non-finite targets and exact ties, $q_w = q_u$, without an exclusion margin or logit rescaling. The trainer refuses non-finite supervision before constructing the loss. Equations (3) and (4) show the list-level form.

- **Sequence supervision (Equation (4)).** The native decoder prepends the projected context token before `BOS`. The sequence loss leaves `BOS` unsupervised but includes the following non-padding tokens, including the engine and regression-task header and `EOS`. That header is fixed during inference. Label smoothing of 0.05 applies over legal tokens; token losses are averaged within each program before the normalized rank-weighted mean over programs. The displayed aggregate coefficients match the native objective; the checkpoint's objective identity and configuration still belong in its evaluation record.
- **Runtime supervision (Equation (5)).** The auxiliary runtime term compares $\ln(1+s)$ with $\ln(1+\mathrm{softplus}(h))$ by Huber loss with threshold 1, for measured seconds $s \geq 0$ and raw scalar head output h.
- **Ranking targets (Equations (3) and (5)).** Here $q_w = 1/(1+\mathrm{RMSE}_w/m_p)$ (Equation (5)), with m_p the corpus list's median RMSE. For microbatch normalization, Bradley–Terry weights use $|q_w - q_u|$, doubled when either member is at or above the 90th percentile. Weights are mean-scaled and cast to float32; weighted losses average non-tied within-comparison pairs in the rank-local microbatch. Listwise losses average eligible groups. The global-batch route uses separate reductions.
- **Registered parameter inventory (Section 6.4).** The historical reference registers 27,796,494 parameters in total (six decoder blocks, width 512, four attention heads, prefix context token and maximum length 128 tokens over the 401-token grammar). Each of the writer π_θ and ranking-map v_ψ token tables allocates 657 rows: 401 grammar rows plus 256 reserved rows. The reserved rows are never indexed as valid input or output tokens; the tied writer projection is restricted to the grammar rows.

  The registered total also includes zero-weight stability and loss-scale terms, so it does not measure learned or exercised capacity. Approximately 6.90 million parameters lie outside the writer's token generation path. Game-day decoding bypasses them, but the native checkpoint loader still requires all of them.
- **Optimization notes (Section 6.5).** The records describe AdamW training (learning rate 10^{-4}, weight decay 0.01, gradient-norm clip 1.0, global batch size 2,048) on eight A100 GPUs in September 2026 over 10,214,332 scored programs from 80 training datasets (7,877 lists; 8,894 contexts). Global batch size alone does not establish per-GPU microbatch or accumulation settings. Checkpoint acceptance evidence and a held-out promotion receipt remain required by the evaluation contract (Section 7.1).
- **Execution sandbox and parity (Section 6.6).** The final evaluation protocol requires candidate isolation with seccomp and Landlock, cgroup memory limits with swap disabled and native versus emitted-code agreement within $10^{-9}\max(1,|\hat{y}|)$. Isolation and parity receipts from the final evaluator belong to the same evaluation record.

D Flywheel Telemetry

Each capture keeps its own date, population scope and event units (Section 6.8). Appendix D.1 gives the canonical adapter-admission chronology.

The exhibits distinguish adapter decisions from program-incumbent events and operating clocks (Appendix D.18). Historical GP diagnostics (Appendix D.19) retain their own separate comparison protocol.

D.1 Retained admission ledger and pre-decision margins

Table 10: **The canonical admission ledger records every complete row in the prefix.** Strict gains and ties use unrounded candidate minus prior-best utility, without outcome selection. Scope: Section 6.8.

Quantity	Recorded value
Capture cutoff (UTC)	2026-11-02 23:00
Complete rounds	775–3950 (3,176 records)
Admissions / rejections	8 / 3,168
Strict-gain rounds	785, 859, 2750, 3175
Exact-tie rounds	782, 882, 2795, 3100
Final retained utility	1.1389572403748425
Recorded best-round identity	3175
New rows since prior capture	753 (0 admitted)
Rejected margins in $[-10^{-4}, 0)$	26
Rejected margins in $[-10^{-3}, -10^{-4})$	51
Rejected margins below -10^{-3}	3,091
Closest rejected margin	−0.0000433838086485 (round 803)
Median rejected margin	−0.0033601045139468

Table 10 collects the chronology, exact counts and margin summaries for one capture. The pre-decision margin is $\Delta_r = u_{\mathrm{cand},r} - u_{\mathrm{prev},r}$; the default rule accepts $\Delta_r \geq 0$.

Figure 6 summarizes complete trailing 50-round windows and cumulative near-tie rejections from that same record.

D.2 Recorded-best trajectory and cumulative acceptance rate

Figure 14 covers 1,889 decisions in rounds 775–2663. From 1.1074, the recorded best reaches 1.1376 at round 785 and 1.1378 at 859; rounds 782 and 882 are accepted ties. The cumulative accepted share reaches 18.18% (2/11) at round 785, 3.70% (4/108) at round 882 and 0.21% (4/1,889) after 1,781 further rejections. Each round contributes once to the utility-shortfall distribution, without counting independent tasks or distinct programs. Cross-harness comparisons require matched data, normalization and scoring protocols.

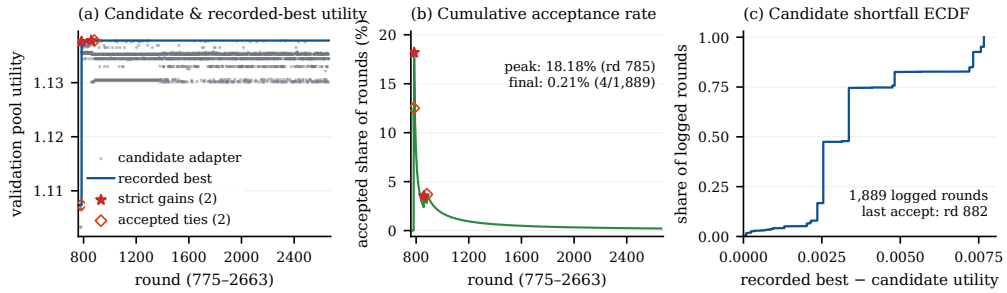


Figure 14: **Cumulative adapter acceptance falls to 0.21% across rounds 775–2663.** All three panels use the same 1,889 consecutive admission evaluations (Table 10). (a) Candidate validation utility (logged scalar score; grey) and post-decision recorded best (blue), with stars at strict gains (rounds 785 and 859) and hollow diamonds at accepted ties (rounds 782 and 882). (b) Cumulative accepted share of elapsed rounds (%), peaking at 18.18% at round 785 and declining across 1,781 subsequent rejected rounds to 0.21% (4/1,889). (c) Empirical cumulative distribution (fraction of evaluations) of recorded-best minus candidate validation utility; zero marks the four accepted adapter evaluations. Scope: Section 6.8.

D.3 Admission frequency through a later cutoff

Figure 15 groups the complete admission journal from the canonical ledger's first round (Table 10) through the latest complete admission matched to a retained summary. Its 24 blocks contain 23 full blocks and one partial block; 19 have no admission, including 17 consecutive full blocks in rounds 975–2674. The 21 available summaries (3145–3165) match the journal's scores and decisions; older rows are journal-only. The last accepted decision, at round 3100, is an exact tie, followed by 65 rejections. Frequencies count decisions, not independent tasks, distinct programs or a stationary admission probability. Stable log-prefix hashes bind this separate capture. This trace retains its own capture cutoff; the field-binding limits in Section 6.8 still apply.

D.4 Candidate and retained-best decisions in a recent capture

The capture in Figure 16 retains the latest 500 consecutive complete admission records matched to the per-round audit. It remains separate from earlier windows: gaps are unjoined and source binding remains unresolved (Section 6.8).

D.5 Rolling adapter admission frequencies

Each trailing window in Figure 17 counts exactly 100 observed decisions, including accepted ties and rejected candidates. Its capture is separate from the canonical ledger in Table 10. These overlapping counts describe decisions on adaptively reused development contexts; the independent first-proposal comparison remains pending.

D.6 Recorded selection changes

Figure 18 uses an observer that refuses changed evaluation identities, inconsistent selection counts and duplicate context identifiers before comparing ordered program identities. It counts candidate and retained differences from one stored reference, rather than counting utility changes. Admission uses unrounded utility and accepts exact ties by default, so the recorded best-round identity can change without a strict score gain. Counts describe changes on reused development contexts; matched controls are still required for attribution to training or proposal quality.

D.7 Recorded retained utility lift

Figure 19 subtracts one fixed recorded incumbent from each post-decision retained score. The prefix uses one logged evaluation identity and includes all completed decisions through its capture cutoff. These adaptive observations retain the source-binding and evaluation limits in Section 6.8.

D.8 Recorded admission frequencies

Figure 20 retains every completed decision in its cumulative denominator. Admission includes exact ties; a separate line counts strict utility gains. Both are descriptive frequencies on reused development contexts (Section 6.8).

D.9 Current adapter admission decisions

Complete entries in the summary, adapter-admission and per-round journals supply the scalar decisions in Figure 21.

A later capture follows the recorded best through the rejection run (Figure 22). Figure 23 counts pool insertions and adapter rejections over the same rounds.

D.10 Aligned event counters

Figure 24 aligns two committed histories sharing logged round identifiers. The consecutive insertion records bracket the full admission window; rounds without a recorded insertion are retained as zero counts. A recorded pool insertion does not imply an adapter admission.

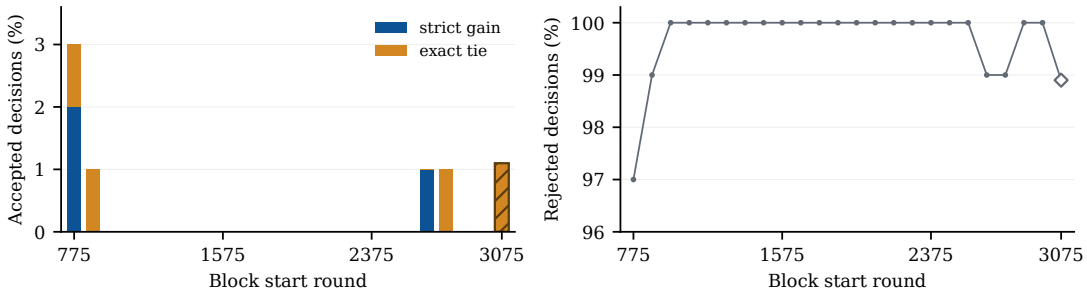


Figure 15: **Nineteen of 24 observed-round blocks record zero adapter admissions.** All 2,391 complete journal decisions in rounds 775–3165, captured at 20:17 UTC on 1 November 2026. Blocks contain 100 consecutive observed rounds; the final 3075–3165 block contains 91. Left: accepted strict gains (blue) and exact ties (orange), stacked percentages. Right: rejected decisions (%; axis starts at 96%). Each denominator is that block's observed rounds; hatching and a diamond mark the partial block. Totals: three strict gains, four ties and 2,384 rejections. These adaptive adapter decisions are distinct from program-incumbent events and independent first-proposal game-day performance. Scope: Section 6.8.

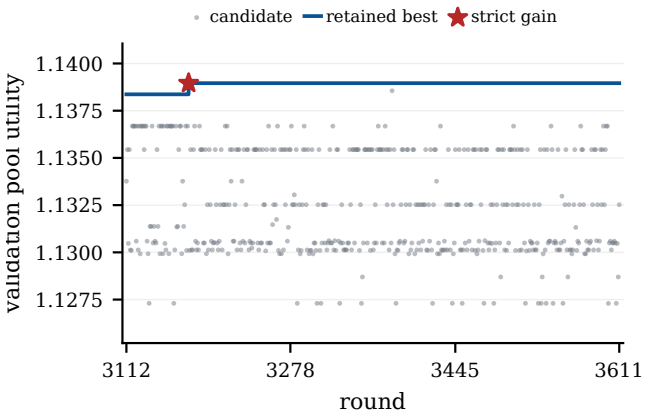


Figure 16: **One strict gain and 499 rejections occur in rounds 3112–3611.** Rounds 3112–3611, captured at 10:25 UTC on 2 November 2026. Grey points show candidate utility; blue is the post-decision retained best (logged dimensionless utility, larger preferred). The star marks the one accepted strict gain, at round 3175; the other 499 decisions are rejections. Classification uses exact logged values, with no rounding margin. These are adaptive decisions on repeatedly used development contexts, not independent first-proposal results or evidence of writer or ranking-map lift. Scope: Section 6.8.

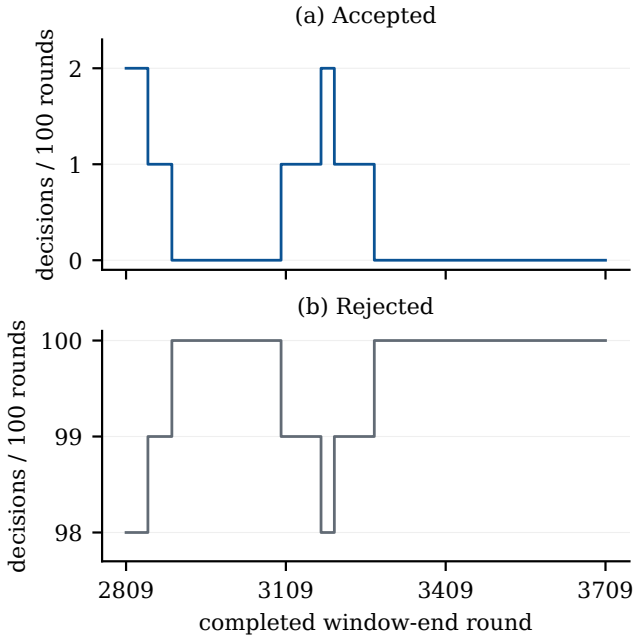


Figure 17: **Trailing 100-round windows record at most two adapter admissions.** The source window contains 1,000 consecutive completed rounds, 2710–3709, captured at 13:30 UTC on 2 November 2026. It records four admissions (two strict gains and two exact ties) and 996 rejections. Panels (a) and (b) show the complementary accepted and rejected counts in each full trailing window, plotted at its final round: 2809–3709, or 901 overlapping windows. The first 99 source rows supply history without a plotted partial window. Every accepted/rejected pair sums to 100; one decision per 100 rounds also equals one percentage point. Decisions use unrounded candidate and prior-best utility. These are adapter admissions, distinct from training-program incumbent events; no uncertainty interval or independent improvement estimate is implied. Scope: Section 6.8.

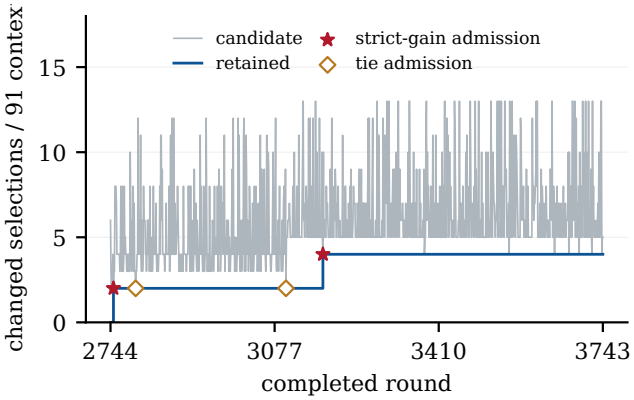


Figure 18: **Retained program selections change only at admitted rounds.** Rounds 2744–3743, captured at 14:34 UTC on 2 November 2026. All 1,000 completed records compare selections on 91 reused development contexts with the recorded round-882 reference. Grey: candidate before admission; blue: post-decision retained selections. Stars: two strict-gain admissions; hollow diamonds: two accepted ties; the other 996 decisions are rejections. These descriptive identity counts do not establish a training effect, writer or ranking-map benefit, or independent proposal quality. Scope: Section 6.8.

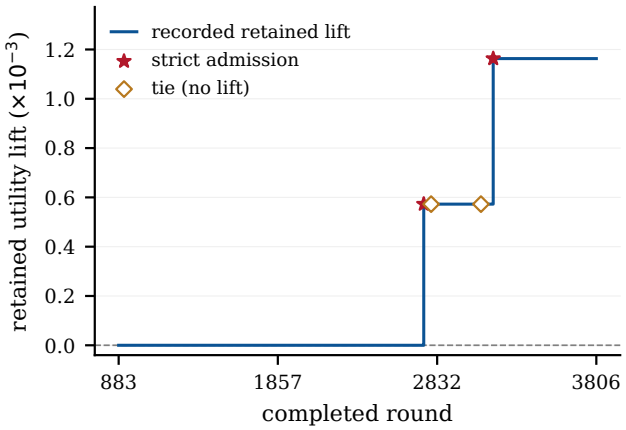


Figure 19: **Retained utility rises in two steps above the round-882 incumbent.** All 2,924 completed records in rounds 883–3806, captured at 17:33 UTC on 2 November 2026. Each step is $\Delta u_r = u_{\text{best},r} - u_{\text{best},882}$, using the same recorded round-882 incumbent throughout. Vertical tick values denote 10^{-3} logged utility units. Stars mark two accepted strict gains; hollow diamonds mark two accepted ties, which add no lift. The other 2,920 decisions are rejections. Classification uses unrounded logged values. Repeated development reuse prevents treating this curve as an independent learning or proposal-quality result. Scope: Section 6.8.

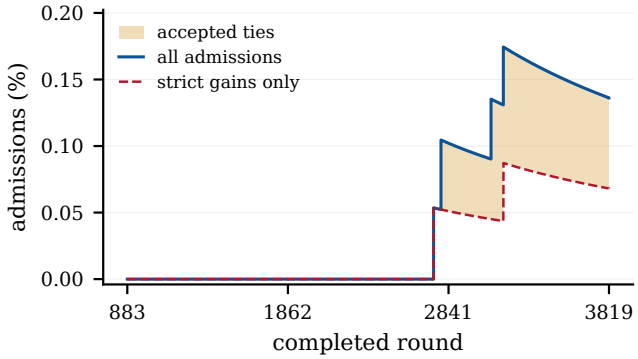


Figure 20: **Four adapter admissions occur across rounds 883–3819.** All 2,937 completed records in rounds 883–3819, captured at 18:27 UTC on 2 November 2026. At round r, the denominator is all $r - 882$ decisions since round 883. Blue counts every admission; dashed red counts strict gains, and the shaded difference counts accepted ties. The final numerator is four: two strict gains and two ties; the other 2,933 decisions are rejections. Classification uses unrounded logged values. These adaptive frequencies do not estimate independent proposal quality or a training effect. Scope: Section 6.8.

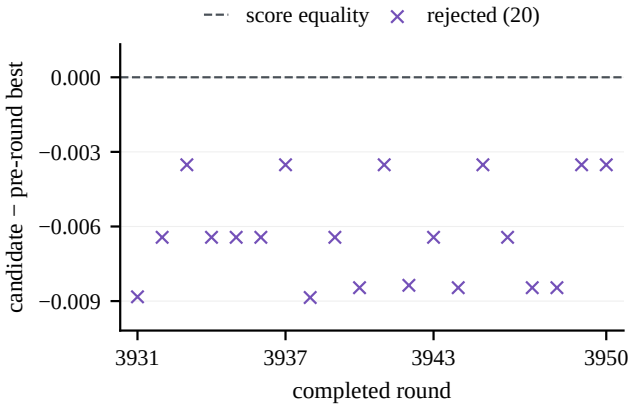


Figure 21: **All 20 candidates in the window are rejected.** Rounds 3931–3950, captured at 23:00 UTC on 2 November 2026. Each point is candidate validation utility minus the pre-decision recorded best; zero denotes equality. Markers show logged admission decisions, with all 20 candidates rejected. Recorded-best utility remained 1.1389572. This adaptive development window does not measure independent first-proposal or serving performance. Scope: Section 6.8.

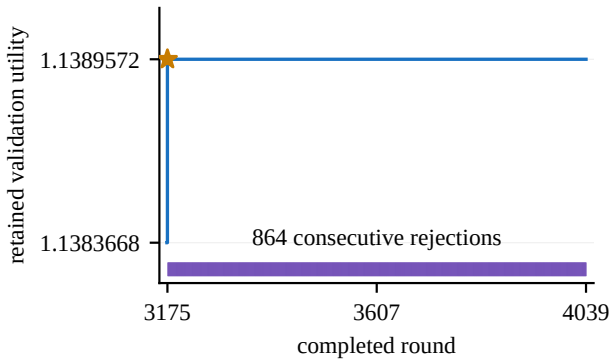


Figure 22: **The recorded best persists through 864 consecutive rejections.** Rounds 3174–4039, captured at 02:24 UTC on 3 November 2026. The star marks the strict gain at round 3175 to 1.1389572403748425; each rug mark is a rejected decision in rounds 3176–4039. Every logged retained score is plotted, without a cumulative maximum. This adaptive plateau is not independent first-proposal or learning-gain evidence. Scope: Section 6.8.

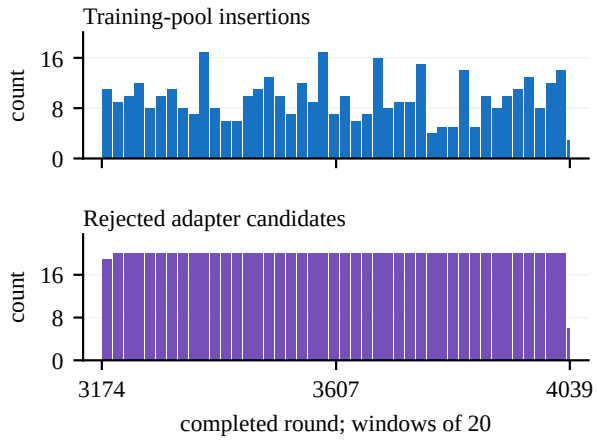


Figure 23: **Pool insertions and adapter rejections count different events.** Rounds 3174–4039, from complete journal prefixes captured on 3 November 2026. Top: 421 logged automatic training-pool insertion events. Bottom: 865 rejected adapter candidates out of 866 decisions. Bars sum 20 consecutive completed rounds; the final bar covers six. Program-level insertions (events per window) and adapter rejections (decisions per window) have different units. These adaptive counts do not measure independent first-proposal or learning gains. Scope: Section 6.8.

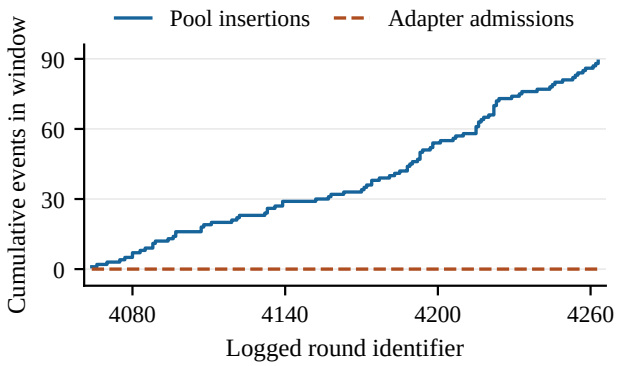


Figure 24: **Pool insertions accumulate without adapter admission.** All 200 admission decisions in rounds 4064–4263, aligned with every program/context insertion in the committed history over those round identifiers. Blue: cumulative pool insertions; orange: cumulative adapter admissions. The window contains 89 insertions and zero adapter admissions. Each count starts at the window boundary and includes all events through the displayed round. These are distinct engineering-event counts, not independent proposal outcomes or evidence of learning gain. Scope: Section 6.8.

D.11 Insertion frequency in fixed round windows

Figure 25 partitions the insertion history in Appendix D.10 into ten consecutive, nonoverlapping windows of 20 logged rounds. Every round contributes to exactly one window; the first window starts at the committed admission history's first round. The denominator is logged rounds, not candidate attempts or elapsed time.

D.12 Program-incumbent promotion margins

Figure 26 isolates the size of a recorded program-incumbent change. Each event compares two programs in its own context; the journal records admitted programs, not the denominator of evaluated or proposed candidates.

D.13 Earlier utility aggregates and operating records

The 25 retained summaries in Figure 27 have a train–validation utility gap of 0.282–0.290. Separate contexts prevent attributing a cause or inferring missing-summary behavior; neither the plateau nor these tables establish transfer, encoder integrity or parameter updates without separate receipts.

Retained operating windows. The separate 20-round event slices record 15 events in 2005–2024, 10 in 2082–2101, 22 in 2343–2362, 15 in 2508–2527 and 21 in 2644–2663, all with zero adapter admissions. The 2696–2715 throughput slice also records none, but reports contexts and time rather than an event total. Appendices D.14, D.16 and D.18 compare these measurements under the clock definitions in Section 6.8.

D.14 Completed-round development trajectories

The four windows in Figure 28 retain separate operating histories. The 2644–2663 trajectory appears in Figure 7, pairing per-round and rolling program-event rates with candidate and recorded-best utility. Figure 29 compares whole-window rates.

D.15 A subsequent development window

The fourth column of Figure 28 shows the 20 summaries in rounds 2508–2527: 15 strict program-incumbent events, zero adapter admissions and retained utility 1.1378 on its original scale. The recorded identity, round 882, is not the first strict gain. Total wall time is 1,731.45 s, giving 41.58 completed rounds and 31.19 events per hour from summed counts and time, not mean per-round rates. Figure 29 compares rates and durations; per-round times remain in the source.

D.16 Program-event distributions and operating rates

Figure 29 compares five separate 20-round windows: 2343–2362, 2508–2527, 2644–2663, 2933–2952 and 3001–3020. Rates divide total events by total completed-round wall time; mean durations divide that time by 20. Sources lacking raw duration or 20 consecutive rounds are excluded, not imputed. Rounds 3001–3020 record 12 program events over 1,809.50 s and no adapter admissions; summary counts match the promotion journal, and scope rules follow Section 6.8.

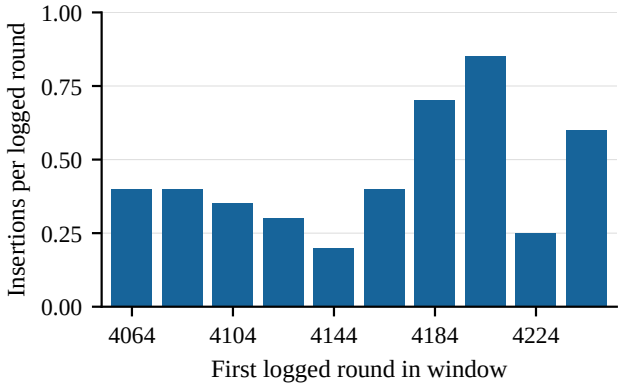


Figure 25: **Pool-insertion frequency varies across fixed round windows.** All ten nonoverlapping 20-round windows in rounds 4064–4263 of the committed history. Each bar is its exact insertion count divided by 20; the horizontal coordinate is the first round in that window. All 89 insertions are retained. These engineering-event rates do not measure per-attempt success, wall-clock throughput or independent learning gain. Scope: Section 6.8.

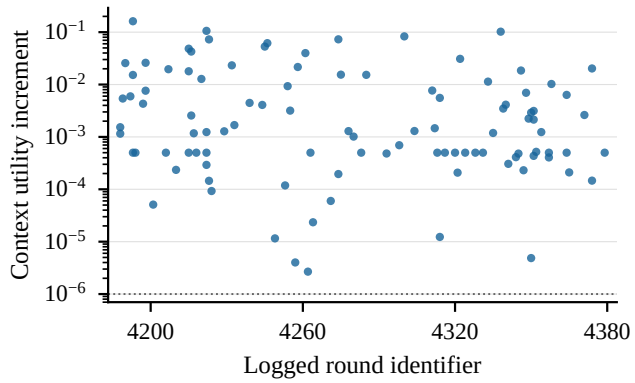


Figure 26: **Program-incumbent promotions have unequal score margins.** All 101 consecutive promotion-journal records in rounds 4188–4379, captured at 14:53 UTC on 3 November 2026. Each point subtracts the logged prior utility from the new utility in that record's own training context; larger utility is preferred. All records passed the ordinary program gate. The vertical axis is logarithmic; the dotted line marks its 10^{-6} margin. Positive increments are required by admission, so this conditional sample does not estimate proposal-level success, adapter improvement or writer learning. Scope: Section 6.8.

D.17 A later program-event window

Event counts and durations measure different parts of a completed round (Figure 30). In this 16:51 UTC capture on 1 November 2026, all 20 summary event counts match the promotion journal and their admission records, separate from Table 10. Pricing a game-day proposal still requires the full attempt ledger (Section 5.6).

D.18 Throughput and program-incumbent diagnostics

The 20-round 2696–2715 slice in Figure 8, captured at 06:54 UTC on 1 November 2026, has no adapter admission. Summary and admission journals agree on candidate utility,

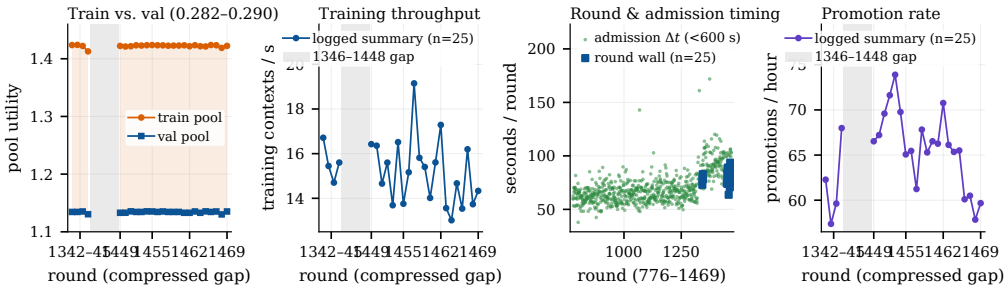


Figure 27: **Earlier retained summaries maintain a 0.282–0.290 train–validation utility gap.** First, second and fourth panels: 25 retained summaries spanning rounds 1342–1345 and 1449–1469 (rounds 1346–1448 were not retained), showing train versus validation pool utility (gap 0.282–0.290), distinct training contexts per completed-round second and strict training-program promotions per hour over up to 20 completed rounds including qualification waits. Third panel: consecutive admission-timestamp intervals: all 695 admission rows (775–1469) carry timestamps, giving 694 consecutive pairs. The panel plots the 692 positive differences below 600 s (green), excluding two longer pauses, alongside the 25 measured round wall times (blue squares). Scope: Section 6.8.

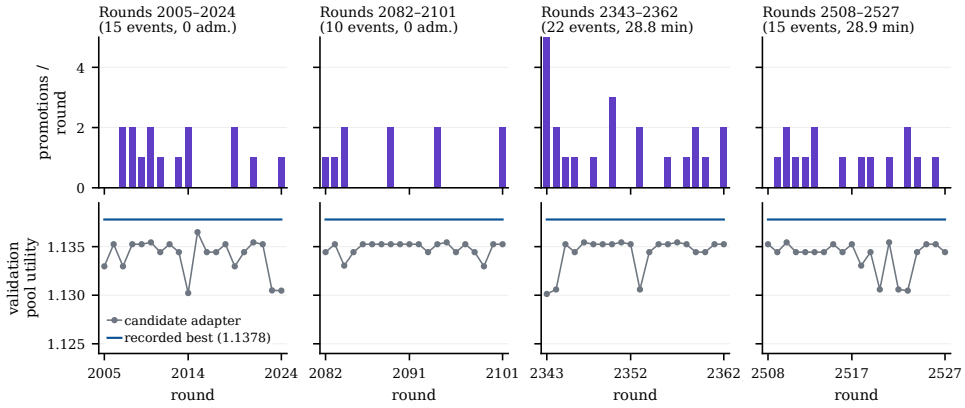


Figure 28: **Four 20-round windows record program promotions with zero adapter admissions.** Columns show four separate 20-round summary windows: rounds 2005–2024 (15 events), 2082–2101 (10 events), 2343–2362 (22 events over 28.8 min of completed-round wall time) and 2508–2527 (15 events over 28.9 min). Top row: strict training-program incumbent promotions per completed round. Bottom row: candidate adapter validation pool utility (grey) and the post-decision recorded best (blue, 1.1378 from round 882). Every window records zero adapter admissions. The separate 2644–2663 window is shown once in Figure 7. Scope: Section 6.8.

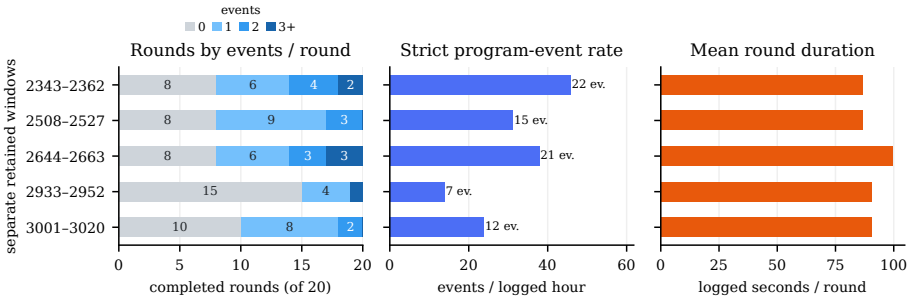


Figure 29: **Program-event rates and round durations vary across five retained 20-round windows.** Each row is a separate capture of 20 consecutive completed rounds with logged wall times. Left: completed rounds grouped by 0, 1, 2 or 3+ strict training-program incumbent events. Center: strict training-program incumbent events divided by total logged round time (events per hour), with total window event counts annotated. Right: mean completed-round duration (seconds) over the 20 rounds, including all within-round work and waits. Windows are displayed separately; program events are distinct from adapter admissions. Scope: Section 6.8.

recorded best, acceptance and best-round identity. Figure 31 adds a later capture. Completed-round throughput divides distinct training contexts by full round time; it differs from presentations per training second and does not count distinct tasks across rounds.

The increments in Figure 32 are logged utility changes, not Elo changes or independent performance estimates. The event counts in Figure 33 are checked against the promotion journal.

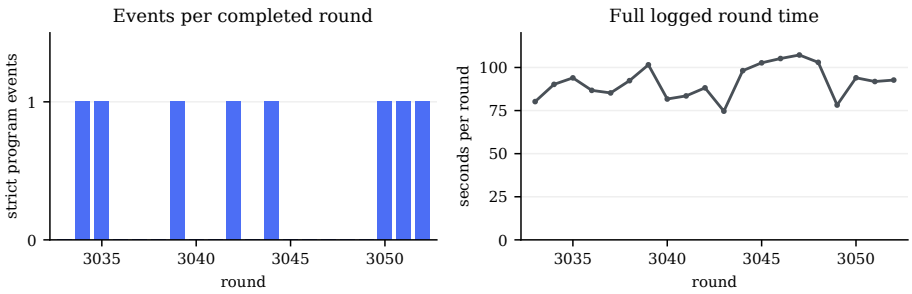


Figure 30: **Rounds 3033–3052 record eight program events and no adapter admissions.** Every completed summary in rounds 3033–3052, including zero-event rounds. Left: strict training-program incumbent events per completed round (count). Right: each round's full logged wall time (seconds), including work and waits. The window records eight events over 1,831.32 s (15.73 events per hour) and no adapter admissions. These are distinct event and time measures. Scope: Section 6.8.

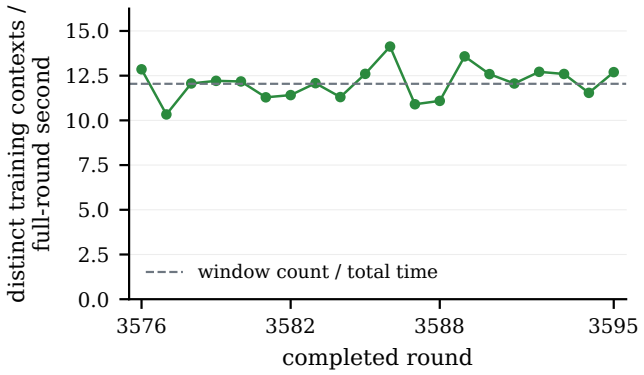


Figure 31: **Completed-round throughput varies across rounds 3576–3595.** Twenty consecutive rounds, 3576–3595, captured at 09:54 UTC on 2 November 2026. Each point divides 1,219 distinct training contexts by that round's full wall time (contexts per second), including all work and waits. The dashed line divides the window's summed counts by summed time. Repeated contexts across rounds are not independent tasks; this is an operating observation, not deployment throughput. Scope: Section 6.8.

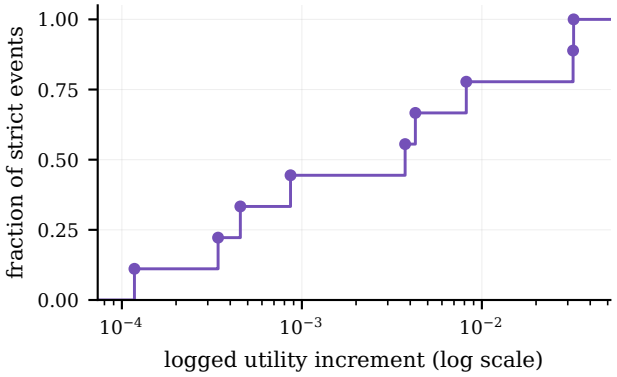


Figure 32: **Nine strict program events occur across rounds 3656–3675.** Rounds 3656–3675, captured at 12:25 UTC on 2 November 2026. The empirical cumulative distribution gives each of the nine strict events equal weight; the horizontal axis is logarithmic. An increment is the journaled new utility minus its previous incumbent utility, computed from unrounded values. All 20 completed rounds were checked against the journal; 13 had no event and contribute no zero-gain observation. The window records zero adapter admissions. Event-level confidence intervals are unavailable; the curve is descriptive, without uncertainty bands. Scope: Section 6.8.

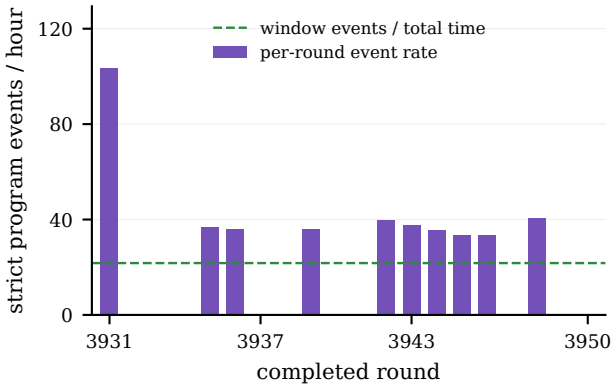


Figure 33: **Twelve strict program events occur across rounds 3931–3950.** Rounds 3931–3950, captured at 23:00 UTC on 2 November 2026. Bars divide each round's event count by its full wall time (events per hour); the dashed line divides the 12 events by the total 1,987.27 s (21.74 events per hour). All 20 rounds are retained, including 10 with no event. The window records zero adapter admissions and no retained-utility gain; these program events do not measure independent first-proposal or game-day serving performance. Scope: Section 6.8.

D.19 Historical GP search diagnostics

The 159 repeated dataset/cut/arm groups in Figure 34 span 55 dataset/cut pairs; 64 groups with lower final RMSE cover 23 pairs. Shared incumbent and teacher seeds make the search arms mutually dependent.

The remaining three panels of Figure 34 show one dataset/-cut chosen post hoc for the largest policy-seeded relative gain. Every generation is shown without a cumulative minimum.

All arms finish at the same value; the policy-seeded arm first reaches it one generation after the controls. These are not independent discoveries (Section 6.8).

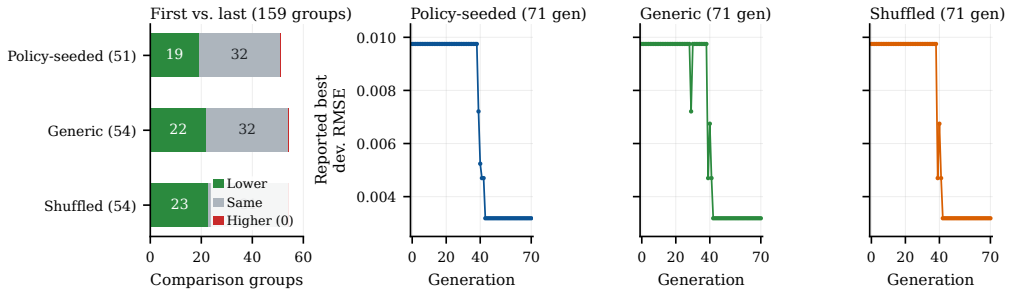


Figure 34: **Historical GP arms reach the same final RMSE on the post-hoc trajectory.** First panel: first-to-last reported best development RMSE across all 159 repeated comparison groups (55 distinct dataset/cut pairs) at 1,000 evaluations per search, grouped by search arm. Second through fourth panels: all 71 generations per arm for one historical dataset/cut selected post hoc for the largest policy-seeded relative improvement. RMSE is in historical evaluator units (lower is better). The arms shared incumbent and teacher seeds, and elapsed time varied; these development records do not establish independent discoveries or a learned-policy advantage. Scope: Section 6.8.

References

[1] T. Anthony, Z. Tian, and D. Barber. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems*, volume 30, pages 5360–5370, 2017. doi: 10.48550/arXiv.1705.08439. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/d8e1344e27a5b08cdfd5d027d9b8d6de-Paper.pdf.

[2] P. Azoulay, C. Fons-Rosen, and J. S. Graff Zivin. Does science advance one funeral at a time? *American Economic Review*, 109(8):2889–2920, 2019. doi: 10.1257/aer.20161574.

[3] L. Bailey, K. Wen, K. Dong, T. Hashimoto, and T. Ma. Scaling self-play with self-guidance. arXiv:2604.20209, 2026. URL https://arxiv.org/abs/2604.20209.

[4] D. Balduzzi, K. Tuyls, J. Pérolat, and T. Graepel. Re-evaluating evaluation. In *Advances in Neural Information Processing Systems*, volume 31, pages 3268–3279, 2018. doi: 10.48550/arXiv.1806.02643. URL https://arxiv.org/abs/1806.02643.

[5] M. Balog, A. L. Gaunt, M. Brockschmidt, S. Nowozin, and D. Tarlow. DeepCoder: Learning to write programs. In *International Conference on Learning Representations (ICLR)*, 2017. doi: 10.48550/arXiv.1611.01989. URL https://arxiv.org/abs/1611.01989.

[6] R. Bellman. *Dynamic Programming*. Princeton University Press, Princeton, NJ, 1957. doi: 10.1515/9781400835386.

[7] J. Bennett and S. Lanning. The Netflix Prize. In *Proceedings of the KDD Cup and Workshop 2007*, 2007. URL https://www.cs.uic.edu/~liub/KDD-cup-2007/proceedings/The-Netflix-Prize-Bennett.pdf.

[8] N. Bloom, C. I. Jones, J. Van Reenen, and M. Webb. Are ideas getting harder to find? *American Economic Review*, 110(4):1104–1144, 2020. doi: 10.1257/aer.20180338.

[9] M. A. Boden. *The Creative Mind: Myths and Mechanisms*. Weidenfeld & Nicolson, London, 1990. doi: 10.4324/9780203508527.

[10] M. A. Boden. Creativity and artificial intelligence. *Artificial Intelligence*, 103(1–2):347–356, 1998. doi: 10.1016/S0004-3702(98)00055-1.

[11] J. L. Borges. La biblioteca de Babel. In *El jardín de senderos que se bifurcan*. Sur, Buenos Aires, 1941. Printed 30 December 1941, issued 1942 (LCCN 43002084). English translation by James E. Irby, "The Library of Babel", in *Labyrinths*, New Directions, 1962.

[12] R. A. Bradley and M. E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3–4):324–345, 1952. doi: 10.1093/biomet/39.3-4.324.

[13] R. Brandom. What happens when AI starts building itself? TechCrunch, 14 May 2026. URL https://techcrunch.com/2026/05/14/what-happens-when-ai-starts-building-itself/.

[14] T. Brooks, B. Peebles, C. Holmes, W. DePue, Y. Guo, L. Jing, D. Schnurr, J. Taylor, T. Luhman, E. Luhman, C. Ng, R. Wang, and A. Ramesh. Video generation models as world simulators. OpenAI technical report, February 2024. URL https://openai.com/index/video-generation-models-as-world-simulators/.

[15] D. T. Campbell. Blind variation and selective retention in creative thought as in other knowledge processes. *Psychological Review*, 67(6):380–400, 1960. doi: 10.1037/h0040373.

[16] Z. Cao, T. Qin, T.-Y. Liu, M.-F. Tsai, and H. Li. Learning to rank: From pairwise approach to listwise approach. In *Proceedings of the 24th International Conference on Machine Learning (ICML)*, pages 129–136, 2007. doi: 10.1145/1273496.1273513.

[17] G. C. Cawley and N. L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine*

Learning Research, 11(70):2079–2107, 2010. URL https://www.jmlr.org/papers/v11/cawley10a.html.

[18] W. G. Chase and H. A. Simon. Perception in chess. *Cognitive Psychology*, 4(1):55–81, 1973. doi: 10.1016/0010-0285(73)90004-2.

[19] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, Ł. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. W. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. A. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. arXiv:2107.03374, 2021. URL https://arxiv.org/abs/2107.03374.

[20] X. Chen, C. Liang, D. Huang, E. Real, K. Wang, H. Pham, X. Dong, T. Luong, C.-J. Hsieh, Y. Lu, and Q. V. Le. Symbolic discovery of optimization algorithms. In *Advances in Neural Information Processing Systems*, volume 36, pages 49205–49233, 2023. doi: 10.48550/arXiv.2302.06675. URL https://arxiv.org/abs/2302.06675.

[21] Z. Chen, Y. Deng, H. Yuan, K. Ji, and Q. Gu. Self-play fine-tuning converts weak language models to strong language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 6621–6642, 2024. doi: 10.48550/arXiv.2401.01335. URL https://arxiv.org/abs/2401.01335.

[22] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, volume 30, pages 4299–4307, 2017. doi: 10.48550/arXiv.1706.03741. URL https://arxiv.org/abs/1706.03741.

[23] S. Cooper, F. Khatib, A. Treuille, J. Barbero, J. Lee, M. Beenen, A. Leaver-Fay, D. Baker, Z. Popović, and Foldit Players. Predicting protein structures with a multiplayer online game. *Nature*, 466(7307):756–760, 2010. doi: 10.1038/nature09304.

[24] H. Dai, Y. Tian, B. Dai, S. Skiena, and L. Song. Syntax-directed variational autoencoder for structured data. In *International Conference on Learning Representations (ICLR)*, 2018. doi: 10.48550/arXiv.1802.08786. URL https://arxiv.org/abs/1802.08786.

[25] C. Darwin. *On the Origin of Species by Means of Natural Selection, or the Preservation of Favoured Races in the Struggle for Life*. John Murray, London, 1859. doi: 10.5962/bhl.title.68064. URL http://darwin-online.org.uk/content/frameset?itemID=F373&viewtype=text&pageseq=1.

[26] A. D. de Groot. *Thought and Choice in Chess*. Mouton, The Hague, 1965. doi: 10.5117/9789053569986.

[27] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, et al. DeepSeek-V3 technical report. arXiv:2412.19437, 2024. URL https://arxiv.org/abs/2412.19437.

[28] K. Dong and T. Ma. STP: Self-play LLM theorem provers with iterative conjecturing and proving. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 14114–14136, 2025. doi: 10.48550/arXiv.2502.00212. URL https://proceedings.mlr.press/v267/dong25h.html.

[29] D. Donoho. 50 years of data science. *Journal of Computational and Graphical Statistics*, 26(4):745–766, 2017. doi: 10.1080/10618600.2017.1384734.

[30] S. Du, X. Yan, J. Shi, Z. Cao, S. Feng, Z. Liang, B. Sun, T. Peng, Y. Zhou, X. Li, J. Zhou, L. He, B. Zhang, and L. Bai. MLEvolve: A self-evolving framework for automated machine learning algorithm discovery. arXiv:2606.06473, 2026. URL https://arxiv.org/abs/2606.06473.

[31] K. Ellis, C. Wong, M. Nye, M. Sablé-Meyer, L. Morales, L. Hewitt, L. Cary, A. Solar-Lezama, and J. B. Tenenbaum. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pages 835–850, 2021. doi: 10.1145/3453483.3454080.

[32] A. E. Elo. *The Rating of Chessplayers, Past and Present*. Arco, New York, 1978.

[33] A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatain, A. Novikov, F. J. R. Ruiz, J. Schrittwieser, G. Swirszcz, D. Silver, D. Hassabis, and P. Kohli. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930): 47–53, 2022. doi: 10.1038/s41586-022-05172-4.

[34] M. Fey, V. Kocijan, F. Lopez, J. E. Lenssen, and J. Leskovec. KumoRFM: A relational foundation model. Kumo Technical Report, May 2025. URL https://docs.nvidia.com/sdgm/research/concepts-models/kumorfm-paper.

[35] R. M. Friedberg. A learning machine: Part I. *IBM Journal of Research and Development*, 2(1):2–13, 1958. doi: 10.1147/rd.21.0002.

[36] I. J. Good. Speculations concerning the first ultraintelligent machine. In F. L. Alt and M. Rubinoff, editors, *Advances in Computers*, volume 6, pages 31–88. Academic Press, 1966. doi: 10.1016/S0065-2458(08)60418-0.

[37] J. Gottweis, W.-H. Weng, A. Daryin, T. Tu, P. Sirkovic, A. Myaskovsky, G. Glowaty, F. Weissenberger, A. Orlandi, D. Popovici, A. Palepu, K. Rong, R. Tanno, K. Saab, F. Zhang, J. Blum, A. Carroll, K. Kulkarni, N. Tomasev, D. Zverinski, I. Rendulic, E. Vedadi, F. Hasler, L. Rimanic, M. Boia, I. Budiselic, B. Feinstein, M. Bellaiche, T. Sheffer, J. Freyberg, J. Ratcliff, O. Bertolli, K. Chou, A. Hassidim, B. Gokturk, A. Vahdat, Y. Guan, V. Dhillon, E. D. Vaishnav, B. Lee, T. R. D. Costa, J. R. Penadés, G. Peltz, Y. Matias, J. Manyika, D. Hassabis, Y. Xu, P. Kohli, A. Pawlosky, A. Karthikesalingam, and V. Natarajan. Accelerating scientific discovery with Co-Scientist. *Nature*, 655(8122):487–496, 2026. doi: 10.1038/s41586-026-10644-y. Preprint: arXiv:2502.18864 (2025), "Towards an AI co-scientist".

[38] S. Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 317–330, 2011. doi: 10.1145/1926385.1926423.

[39] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.

[40] R. W. Hamming. You and your research. Bell Communications Research Colloquium Seminar, Morris Research and Engineering Center, Morristown, NJ, 1986. URL https://hdl.handle.net/10945/37504. Talk given 7 March 1986; transcribed by J. F. Kaiser.

[41] D. Hassabis and A. Kantrowitz. Google DeepMind CEO Demis Hassabis: The path to AGI, LLM creativity, and Google smart glasses. Big Technology Podcast, 23 January 2025. URL https://www.bigtechnology.com/p/google-deepmind-ceo-demis-hassabis.

[42] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 30016–30030, 2022. doi: 10.48550/arXiv.2203.15556. URL https://arxiv.org/abs/2203.15556.

[43] J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, MI, 1975. doi: 10.7551/mitpress/1090.001.0001.

[44] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025. doi: 10.1038/s41586-024-08328-6.

[45] T. Hubert, R. Mehta, L. Sartran, M. Z. Horváth, G. Žužić, E. Wieser, A. Huang, J. Schrittwieser, Y. Schroecker, H. Masoom, O. Bertolli, T. Zahavy, A. Mandhane, J. Yung, I. Beloshapka, B. Ibarz, V. Veeriah, L. Yu, O. Nash, P. Lezeau, S. Mercuri, C. Sönne, B. Mehta, A. Davies, D. Zheng, F. Pedregosa, Y. Li, I. von Glehn, M. Rowland, S. Albanie, A. Velingker, S. Schmitt, E. Lockhart, E. Hughes, H. Michalewski, N. Sonnerat, D. Hassabis, P. Kohli, and D. Silver. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106): 607–613, 2026. doi: 10.1038/s41586-025-09833-y. Published online 12 November 2025.

[46] B. F. Jones. The burden of knowledge and the "Death of the Renaissance Man": Is innovation getting harder? *Review of Economic Studies*, 76(1): 283–317, 2009. doi: 10.1111/j.1467-937X.2008.00531.x.

[47] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, A. Bridgland, C. Meyer, S. A. A. Kohl, A. J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A. W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.

[48] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. arXiv:2001.08361, 2020. URL https://arxiv.org/abs/2001.08361.

[49] A. Karpathy. `autoresearch`: AI agents running research on single-GPU nanochat training automatically. GitHub repository, 6 March 2026. URL https://github.com/karpathy/autoresearch.

[50] F. Khatib, F. DiMaio, Foldit Contenders Group, Foldit Void Crushers Group, S. Cooper, M. Kazmierczyk, M. Gilski, S. Krzywda, H. Zábranská, I. Pichová, J. Thompson, Z. Popović, M. Jaskolski, and D. Baker. Crystal structure of a monomeric retroviral protease solved by protein folding game

players. *Nature Structural & Molecular Biology*, 18(10):1175–1177, 2011. doi: 10.1038/nsmb.2119.

[51] Kimi Team. Kimi K2: Open agentic intelligence. arXiv:2507.20534, 2025. URL https://arxiv.org/abs/2507.20534.

[52] A. Koestler. *The Act of Creation*. Hutchinson, London, 1964.

[53] J. R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA, 1992. URL https://mitpress.mit.edu/9780262111706/genetic-programming/.

[54] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, pages 1097–1105, 2012. doi: 10.1145/3065386. URL https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks. Republished in *Communications of the ACM*, 60(6):84–90, 2017.

[55] T. S. Kuhn. *The Structure of Scientific Revolutions*. University of Chicago Press, Chicago, 1962. doi: 10.7208/chicago/9780226458144.001.0001.

[56] M. J. Kusner, B. Paige, and J. M. Hernández-Lobato. Grammar variational autoencoder. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1945–1954, 2017. doi: 10.48550/arXiv.1703.01925. URL https://proceedings.mlr.press/v70/kusner17a.html.

[57] T. Lattimore and C. Szepesvári. *Bandit Algorithms*. Cambridge University Press, Cambridge, 2020. doi: 10.1017/9781108571401.

[58] Y. LeCun and A. Kantrowitz. Why can't AI make its own discoveries? — With Yann LeCun. Big Technology Podcast, 19 March 2025. URL https://podscripts.co/podcasts/big-technology-podcast/why-cant-ai-make-its-own-discoveries-with-yann-lecun.

[59] Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553): 436–444, 2015. doi: 10.1038/nature14539.

[60] J. Lee, W. Kladwang, M. Lee, D. Cantu, M. Azizyan, H. Kim, A. Limpaecher, S. Yoon, A. Treuille, R. Das, and EteRNA Participants. RNA design rules from a massive open laboratory. *Proceedings of the National Academy of Sciences*, 111(6):2122–2127, 2014. doi: 10.1073/pnas.1313039111.

[61] Y.-J. Lee, S. Kim, M. Kang, A. C. L. Chuen, Z. Chen, S. Han, T. Jung, and D. Kang. Evolution Fine-Tuning: Learning to discover across 371 optimization tasks. arXiv:2606.29082, 2026. URL https://arxiv.org/abs/2606.29082.

[62] J. Lehman and K. O. Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223, 2011. doi: 10.1162/EVCO_a_00025.

[63] D. B. Lenat. *AM: An Artificial Intelligence Approach to Discovery in Mathematics as Heuristic Search*. PhD thesis, Stanford University, 1976. Stanford AI Lab Memo AIM-286; report STAN-CS-76-570.

[64] D. B. Lenat. EURISKO: A program that learns new heuristics and domain concepts. The nature of heuristics III: Program design and results. *Artificial Intelligence*, 21(1–2):61–98, 1983. doi: 10.1016/S0004-3702(83)80005-8.

[65] C. Lu, C. Lu, R. T. Lange, Y. Yamada, S. Hu, J. Foerster, D. Ha, and J. Clune. Towards end-to-end automation of AI research. *Nature*, 651 (8107):914–919, 2026. doi: 10.1038/s41586-026-10265-5. Preprint: arXiv:2408.06292 (2024), "The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery".

[66] D. J. Mankowitz, A. Michi, A. Zhernov, M. Gelmi, M. Selvi, C. Paduraru, E. Leurent, S. Iqbal, J.-B. Lespiau, A. Ahern, T. Köppe, K. Millikin, S. Gaffney, S. Elster, J. Broshear, C. Gamble, K. Milan, R. Tung, M. Hwang, T. Cemgil, M. Barekatain, Y. Li, A. Mandhane, T. Hubert, J. Schrittwieser, D. Hassabis, P. Kohli, M. Riedmiller, O. Vinyals, and D. Silver. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618(7964):257–263, 2023. doi: 10.1038/s41586-023-06004-9.

[67] H. Massalin. Superoptimizer: A look at the smallest program. In *Proceedings of the 2nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS II)*, pages 122–126, 1987. doi: 10.1145/36206.36194.

[68] A. Merchant, S. Batzner, S. S. Schoenholz, M. Aykol, G. Cheon, and E. D. Cubuk. Scaling deep learning for materials discovery. *Nature*, 624(7990): 80–85, 2023. doi: 10.1038/s41586-023-06735-9.

[69] C. Metz. In two moves, AlphaGo and Lee Sedol redefined the future. Wired, 16 March 2016. URL https://www.wired.com/2016/03/two-moves-alphago-lee-sedol-redefined-future/.

[70] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.

[71] D. J. Montana. Strongly typed genetic programming. *Evolutionary Computation*, 3(2):199–230, 1995. doi: 10.1162/evco.1995.3.2.199.

[72] J.-B. Mouret and J. Clune. Illuminating search spaces by mapping elites. arXiv:1504.04909, 2015. URL https://arxiv.org/abs/1504.04909.

[73] A. Müller, C. Curino, and R. Ramakrishnan. MotherNet: Fast training and inference via hyper-network transformers. In *International Conference on Learning Representations (ICLR)*, 2025. doi: 10.48550/arXiv.2312.08598. URL https://arxiv.org/abs/2312.08598.

[74] J. Nielsen and J. D. Keasling. Engineering cellular metabolism. *Cell*, 164 (6):1185–1197, 2016. doi: 10.1016/j.cell.2016.02.004.

[75] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131, 2025. URL https://arxiv.org/abs/2506.13131.

[76] NVIDIA. NVIDIA H100 Tensor Core GPU: Specifications. NVIDIA Data Center Documentation, 2026. URL https://www.nvidia.com/en-us/data-center/h100/.

[77] J. Oh, G. Farquhar, I. Kemaev, D. A. Calian, M. Hessel, L. Zintgraf, S. Singh, H. van Hasselt, and D. Silver. Discovering state-of-the-art reinforcement learning algorithms. *Nature*, 648(8093):312–319, 2025. doi: 10.1038/s41586-025-09761-x.

[78] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022. doi: 10.48550/arXiv.2203.02155. URL https://arxiv.org/abs/2203.02155.

[79] M. Park, E. Leahey, and R. J. Funk. Papers and patents are becoming less disruptive over time. *Nature*, 613(7942):138–144, 2023. doi: 10.1038/s41586-022-05543-x.

[80] D. Patel and D. Amodei. Dario Amodei (Anthropic CEO) — The hidden pattern behind every AI breakthrough. Dwarkesh Podcast, 8 August 2023. URL https://www.dwarkesh.com/p/dario-amodei.

[81] C. S. Peirce. *Collected Papers of Charles Sanders Peirce, Volume V: Pragmatism and Pragmaticism*. Harvard University Press, Cambridge, MA, 1934. Edited by Charles Hartshorne and Paul Weiss.

[82] B. Pfahringer, H. Bensusan, and C. Giraud-Carrier. Meta-learning by landmarking various learning algorithms. In *Proceedings of the 17th International Conference on Machine Learning (ICML)*, pages 743–750. Morgan Kaufmann, 2000. URL https://ml.cms.waikato.ac.nz/publications/2000/landmarking_icml2000.pdf.

[83] H. Poincaré. Mathematical creation. *The Monist*, 20(3):321–335, 1910. doi: 10.1093/monist/20.3.321. Translated by George Bruce Halsted.

[84] M. Polanyi. *The Tacit Dimension*. Doubleday, Garden City, NY, 1966.

[85] G. Pólya. *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, Princeton, NJ, 1945. doi: 10.1515/9781400828678.

[86] K. R. Popper. *The Logic of Scientific Discovery*. Hutchinson, London, 1959. doi: 10.4324/9780203994627.

[87] K. R. Popper. *Conjectures and Refutations: The Growth of Scientific Knowledge*. Routledge & Kegan Paul, London, 1963. doi: 10.4324/9780203538074.

[88] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763, 2021. doi: 10.48550/arXiv.2103.00020. URL https://proceedings.mlr.press/v139/radford21a.html.

[89] E. Real, C. Liang, D. R. So, and Q. V. Le. AutoML-Zero: Evolving machine learning algorithms from scratch. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 8007–8019, 2020. doi: 10.48550/arXiv.2003.03384. URL https://proceedings.mlr.press/v119/real20a.html.

[90] J. R. Rice. The algorithm selection problem. In M. Rubinoff and M. C. Yovits, editors, *Advances in Computers*, volume 15, pages 65–118. Academic Press, New York, 1976. doi: 10.1016/S0065-2458(08)60520-3.

[91] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.

[92] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015. doi: 10.1007/s11263-015-0816-y.

[93] A. L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959. doi: 10.1147/rd.33.0210.

[94] J. Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. arXiv:cs/0309048, 2003. URL https://arxiv.org/abs/cs/0309048.

[95] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020. doi: 10.1038/s41586-020-03051-4.

[96] C. E. Shannon. Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41 (314):256–275, 1950. doi: 10.1080/14786445008521796.

[97] M. Shin, J. Kim, B. van Opheusden, and T. L. Griffiths. Superhuman artificial intelligence can improve human decision-making by increasing novelty. *Proceedings of the National Academy of Sciences*, 120(12): e2214840120, 2023. doi: 10.1073/pnas.2214840120.

[98] C. Si, D. Yang, and T. Hashimoto. Can LLMs generate novel research ideas? A large-scale human study with 100+ NLP researchers. In *International Conference on Learning Representations (ICLR)*, 2025. doi: 10.48550/arXiv.2409.04109. URL https://openreview.net/forum?id=M23dTGWCZy.

[99] C. Si, T. Hashimoto, and D. Yang. The ideation–execution gap: Execution outcomes of LLM-generated versus human research ideas. In *International Conference on Learning Representations (ICLR)*, 2026. doi: 10.48550/arXiv.2506.20803. URL https://openreview.net/forum?id=Fllp8l6Puy.

[100] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. doi: 10.1038/nature16961.

[101] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550(7676): 354–359, 2017. doi: 10.1038/nature24270.

[102] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018. doi: 10.1126/science.aar6404. URL https://arxiv.org/abs/1712.01815. Earlier version: arXiv:1712.01815 (2017). Training statistics: supplementary Table S3.

[103] D. K. Simonton. Creativity and discovery as blind variation: Campbell's (1960) BVSR model after the half-century mark. *Review of General Psychology*, 15(2):158–174, 2011. doi: 10.1037/a0022912.

[104] R. Socher. AI's biggest breakthrough won't be a chatbot. Next Big Idea Club, Book Bites, 8 October 2026. URL https://nextbigideaclub.com/magazine/ais-biggest-breakthrough-wont-chatbot-bookbite/61534/.

[105] R. Socher. *The Eureka Machine: Why AI Is the Key to Unlocking a New Era of Scientific Discoveries*. PublicAffairs, New York, 2026. ISBN 9781541705708. URL https://www.hachettebookgroup.com/titles/richard-socher/the-eureka-machine/9781541705708/.

[106] J. M. Stokes, K. Yang, K. Swanson, W. Jin, A. Cubillos-Ruiz, N. M. Donghia, C. R. MacNair, S. French, L. A. Carfrae, Z. Bloom-Ackermann, V. M. Tran, A. Chiappino-Pepe, A. H. Badran, I. W. Andrews, E. J. Chory, G. M. Church, E. D. Brown, T. S. Jaakkola, R. Barzilay, and J. J. Collins. A deep learning approach to antibiotic discovery. *Cell*, 180(4):688–702.e13, 2020. doi: 10.1016/j.cell.2020.01.021.

[107] A. Surina, A. Mansouri, L. Quaedvlieg, A. Seddas, M. Viazovska, E. Abbe, and C. Gulcehre. Algorithm discovery with LLMs: Evolutionary search meets reinforcement learning. arXiv:2504.05108, 2025. URL https://arxiv.org/abs/2504.05108.

[108] R. S. Sutton. The bitter lesson. Essay, 13 March 2019. URL http://www.incompleteideas.net/IncIdeas/BitterLesson.html.

[109] G. Tesauro. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3):58–68, 1995. doi: 10.1145/203330.203343.

[110] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024. doi: 10.1038/s41586-023-06747-5.

[111] A. van den Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding. arXiv:1807.03748, 2018. URL https://arxiv.org/abs/1807.03748.

[112] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017. doi: 10.48550/arXiv.1706.03762. URL https://arxiv.org/abs/1706.03762.

[113] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J. P. Agapiou, M. Jaderberg, A. S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky, J. Molloy, T. L. Paine, C. Gulcehre, Z. Wang, T. Pfaff, Y. Wu, R. Ring, D. Yogatama, D. Wünsch, K. McKinney, O. Smith, T. Schaul, T. Lillicrap, K. Kavukcuoglu, D. Hassabis, C. Apps, and D. Silver. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019. doi: 10.1038/s41586-019-1724-z.

[114] S. Von Arx, L. Chan, and B. Barnes. Recent frontier models are reward hacking. METR blog, 5 June 2025. URL https://metr.org/blog/2025-06-05-recent-reward-hacking/.

[115] G. Wallas. *The Art of Thought*. Jonathan Cape, London, 1926.

[116] K. Wiggers. Sakana walks back claims that its AI can dramatically speed up model training. TechCrunch, 21 February 2025. URL https://techcrunch.com/2025/02/21/sakana-walks-back-claims-that-its-ai-can-dramatically-speed-up-model-training/.

[117] T. Wolf. The Einstein AI model. Technical blog essay, February 2025. URL https://thomwolf.io/blog/scientific-ai.html.

[118] D. H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996. doi: 10.1162/neco.1996.8.7.1341.

[119] M. Yuksekgonul, D. Koceja, X. Li, F. Bianchi, J. McCaleb, X. Wang, J. Kautz, Y. Choi, J. Zou, C. Guestrin, and Y. Sun. Learning to discover at test time. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. doi: 10.48550/arXiv.2601.16175. URL https://arxiv.org/abs/2601.16175.

[120] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai. Self-Taught Optimizer (STOP): Recursively self-improving code generation. In *Conference on Language Modeling (COLM)*, 2024. doi: 10.48550/arXiv.2310.02304. URL https://arxiv.org/abs/2310.02304.

[121] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. arXiv:2505.22954, 2025. URL https://arxiv.org/abs/2505.22954.

[122] A. Zhao, Y. Wu, Y. Yue, T. Wu, Q. Xu, M. Lin, S. Wang, Q. Wu, Z. Zheng, and G. Huang. Absolute Zero: Reinforced self-play reasoning with zero data. arXiv:2505.03335, 2025. URL https://arxiv.org/abs/2505.03335.

[123] Z. Zhong, I. Segal, I. Bercovich, S. Saxena, K. Zhang, and A. Raghunathan. Hardening agent benchmarks with adversarial hacker-fixer loops. arXiv:2606.08960, 2026. URL https://arxiv.org/abs/2606.08960.

[124] B. Zoph and Q. V. Le. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2017. doi: 10.48550/arXiv.1611.01578. URL https://arxiv.org/abs/1611.01578.